TASK 12

Kubernetes Cluster Setup and Deployment

Name: Monu Ajay Rai

Student ID: 5783524

Roll no: 27

Subject: Deep Learning from Production to Deployment

GitHub URL: https://github.com/monu8097/TASK-12-Kubernetes-Cluster-Setup-and-Deployment.git

Table of Contents

1. Objective
2. Tools Used
3. Introduction
4. System Architecture
5. Environment Setup
6. Minikube Cluster Setup
7. Docker Image
8. Deployment YAML
9. Service YAML
10. Kubernetes Deployment
11. Pod Verification
12. Service Verification
13. API Testing
14. Prediction Output
15. Monitoring
16. Results
17. Observations
18. Conclusion
19. Screenshots
20. Appendix

1. Objective

1.1 Task Objective

The objective of Task 12 is to understand Kubernetes orchestration and to deploy containerized applications on a local Kubernetes cluster created with Minikube or Kind. The task builds directly on the previous containerization work: the CIFAR-10 image classification application, which already runs as two Docker containers (a Flask REST API and a Streamlit frontend), is now orchestrated by Kubernetes instead of Docker Compose.

Where Docker Compose starts containers on a single machine from a single file, Kubernetes introduces a declarative control plane. The engineer describes the desired state of the system in YAML manifests, and Kubernetes continuously works to make the actual state of the cluster match that description. This task is therefore as much about learning the vocabulary and mechanics of Kubernetes (Pods, Deployments, Services, ReplicaSets, nodes, namespaces) as it is about running the application.

1.2 Approach Prescribed in the Task

The task statement prescribes five steps. Each of them was carried out in this project and each is documented with evidence in the following sections.

Table 1: Mapping of task steps to implementation

Prescribed Step	How It Was Implemented	Report Section
Set up a local Kubernetes cluster	Minikube started with the Docker driver; node verified as Ready	6, 7
Create deployment manifests	Four YAML files: two Deployments and two Services	9, 10
Deploy sample containers	Flask API and Streamlit images built inside Minikube and deployed with kubectl apply	8, 11
Verify pod and service operations	kubectl get pods / services, health endpoint, API predictions	12, 13, 14, 15
Monitor deployment status	kubectl get deployments / all, rollout restart and rollout status	16

1.3 Deliverables

The task requires three deliverables, all of which are covered by this submission:

- **Kubernetes YAML files:** flask-deployment.yaml, flask-service.yaml, streamlit-deployment.yaml and streamlit-service.yaml (reproduced in Sections 9, 10 and the Appendix).
- **Deployment screenshots:** fourteen terminal and browser screenshots capturing cluster state, pod status, service configuration and predictions (Section 20).
- **Cluster setup report:** this document, which explains the setup, configuration, verification and observations in detail.

1.4 Learning Outcomes

After completing this task, the following competencies are demonstrated:

- Explaining what Kubernetes is and why container orchestration is needed.
- Creating and starting a single-node Kubernetes cluster with Minikube.
- Building container images inside the Minikube environment so that the cluster can use them without an external registry.
- Writing Deployment and Service manifests with correct apiVersion, kind, metadata, selectors, labels and ports.
- Choosing an appropriate Service type (ClusterIP versus NodePort) for internal and external access.
- Using Kubernetes service names for pod-to-pod communication through cluster DNS.
- Verifying and monitoring a deployment with kubectl commands.
- Testing a deployed machine learning inference service end to end.

1.5 Evaluation Criteria

The task will be evaluated on three criteria: deployment success, YAML configuration accuracy and operational understanding. This report addresses each one explicitly:

Table 2: Evaluation criteria and supporting evidence

Criterion	Evidence Provided
Deployment success	Both Pods in Running state (1/1 Ready, 0 restarts); both Deployments 1/1 available; Services created; successful predictions
YAML configuration accuracy	Selectors match Pod labels; image names match the images in Minikube; ports match container ports; Service names match the URL used by Streamlit
Operational understanding	Explanation of each resource, of Service types, of imagePullPolicy, of the rollout restart and of the ReplicaSet history visible in kubectl get all

2. Tools Used

This section lists every tool, framework and library used in the project. The tools fall into four groups: orchestration tools, containerization tools, application frameworks and the development environment.

2.1 Summary Table

Table 3: Tools and technologies used

Tool / Technology	Category	Role in This Project
Kubernetes	Orchestration	Container orchestration platform that manages Pods, Deployments and Services
Minikube	Orchestration	Creates a local single-node Kubernetes cluster (Docker driver)
kubectl	Orchestration	Command-line client used to apply manifests and inspect resources
Docker Desktop / Docker	Containerization	Container runtime; host for the Minikube node container; used to build images
Python	Application	Programming language for the Flask API and Streamlit app
Flask	Application	Lightweight web framework serving the REST prediction API
Streamlit	Application	Framework for the interactive web frontend
TensorFlow / Keras	Application	Loads the trained CIFAR-10 CNN model and runs inference
NumPy	Application	Array handling for image preprocessing and prediction vectors
Pillow (PIL)	Application	Image decoding, RGB conversion and resizing
requests	Application	HTTP client used by Streamlit to call the Flask API
CIFAR-10 CNN model (.keras)	Model	Trained convolutional neural network file cifar10_cnn.keras
YAML	Configuration	Format of the Kubernetes manifests
Windows PowerShell	Environment	Terminal in which all commands were executed
Visual Studio Code	Environment	Editor used for source code and manifests

2.2 Kubernetes

Kubernetes is an open-source container orchestration platform originally designed by Google and now maintained by the Cloud Native Computing Foundation. It automates the deployment, scaling and management of containerized applications. A Kubernetes cluster consists of a control plane, which makes scheduling and management decisions, and one or more worker nodes, which run the containers. The smallest deployable unit is the Pod, which wraps one or more containers that share networking and storage.

Key capabilities that make Kubernetes valuable for machine learning inference services include self-healing (failed Pods are replaced automatically), declarative configuration, service discovery through DNS, rolling updates and the ability to scale the number of replicas by changing a single number.

2.3 Minikube

Minikube is a tool that runs a single-node Kubernetes cluster on a local workstation. It is intended for learning and development. In this project Minikube was started with the Docker driver, which means the Kubernetes node itself runs as a Docker container on Docker Desktop. The node acts as both control plane and worker, which is why the `kubectl get nodes` output shows the role `control-plane` for the single node named `minikube`.

Minikube also bundles convenient features that were used in this task: `minikube image build` builds an image directly into the node, `minikube image ls` lists the images available to the cluster and `minikube service <name> --url` produces a reachable URL for a NodePort service.

2.4 Kind as an Alternative

The task allows either Minikube or Kind (Kubernetes in Docker). Both tools create local clusters for development. Kind runs each node as a Docker container and is frequently used in continuous integration pipelines, whereas Minikube offers a richer set of local conveniences such as add-ons, image build commands and service URL helpers. Minikube was chosen for this project because of its image build integration and its service tunnelling feature, which simplified access to the Streamlit NodePort service on Windows.

Table 4: Comparison of Minikube and Kind

Aspect	Minikube	Kind
Node implementation	VM or container (Docker driver used here)	Docker containers
Typical use	Local learning and development	Testing and CI pipelines
Image loading	minikube image build / load	kind load docker-image
Service access helper	minikube service --url	Port mapping or port-forward
Add-ons	Built-in add-on system	Manual installation

2.5 kubectl

`kubectl` is the official command-line tool for communicating with the Kubernetes API server. Every action in this report, such as applying a manifest, listing Pods or forwarding a port, was performed through kubectl. The tool reads its connection information from the kubeconfig file, which Minikube configures automatically (the `kubeconfig: Configured` line in the `minikube status` output).

2.6 Docker and Docker Desktop

Docker provides the container runtime and image format. Docker Desktop on Windows hosts the Minikube node container. The application images are described by two Dockerfiles (Dockerfile.flask

and Dockerfile.streamlit), both based on the `python:3.11-slim` base image. The images were built with `minikube image build`, which uses the container engine inside the Minikube node so that the resulting images are immediately visible to Kubernetes.

2.7 Application Libraries

Table 5: Python libraries from requirements.txt

Library	Used In	Purpose
flask	flask_api.py	Defines the / and /predict routes and returns JSON
tensorflow	flask_api.py	tf.keras.models.load_model loads cifar10_cnn.keras; model.predict runs inference
numpy	flask_api.py	Converts the image to an array, normalizes it and finds argmax
pillow	flask_api.py, app.py	Opens, converts (RGB) and resizes images
streamlit	app.py, 1_Image_Prediction.py	Builds the dashboard, file uploader, buttons and result widgets
requests	app.py, 1_Image_Prediction.py	Sends GET and POST requests to the Flask API

3. Introduction

3.1 Background

Deploying a machine learning model involves much more than training it. Once a model such as a convolutional neural network has been saved to a file, it must be wrapped in a service that accepts input, performs preprocessing, runs inference and returns a result. That service must then be packaged so that it runs identically on every machine, must be reachable by clients, and must keep running even when individual processes fail. Containers solve the packaging problem and Kubernetes solves the running-and-managing problem.

This project demonstrates the complete path for a small but realistic example. A CNN trained on the CIFAR-10 dataset is exposed through a Flask REST API. A Streamlit application provides a graphical interface through which a user uploads an image and receives the predicted class and confidence. Both parts are containerized and then deployed on a local Kubernetes cluster created with Minikube.

3.2 From Docker Compose to Kubernetes

In the previous task (containerization), the two application containers were started together with Docker Compose. The docker-compose.yml file defined two services, `flask-api` and `streamlit`, attached them to a shared bridge network named `task11-network` and published ports 5000 and 8501 to the host. This works well on a single machine but does not provide the orchestration features required for production-style operation.

Table 6: Docker Compose versus Kubernetes

Concern	Docker Compose	Kubernetes
Desired state	Declared per service in one file	Declared as Deployments and continuously reconciled
Self-healing	Restart policy only	ReplicaSet recreates failed Pods automatically
Scaling	Manual, single host	Change replicas; scheduler places Pods on nodes
Service discovery	Container / service name on a bridge network	Service objects with stable ClusterIP and DNS names
Updates	Recreate containers	Rolling updates with ReplicaSet history
Scope	Single Docker host	Cluster of nodes

3.3 Project Overview

The final system consists of two Deployments and two Services running in the default namespace of a Minikube cluster:

- **flask-api Deployment** running the image `task12-flask-api:latest` with container port 5000, fronted by the `flask-api-service` Service of type ClusterIP.

- **streamlit Deployment** running the image `task12-streamlit:latest` with container port 8501, fronted by the `streamlit-service` Service of type NodePort.

The Streamlit application reaches the Flask API through the Kubernetes DNS name `flask-api-service` on port 5000, which is configured in app.py as `FLASK_API_URL = "http://flask-api-service:5000"`. This is the key integration point between the two parts of the application and demonstrates Kubernetes service discovery.

3.4 The CIFAR-10 Classification Problem

CIFAR-10 is a widely used benchmark dataset containing 60,000 color images of 32 x 32 pixels in ten classes. The training set has 50,000 images and the test set has 10,000 images. The dashboard in app.py reports a test accuracy of 72.4 percent for the model after 10 epochs of training. The ten classes are listed below in the order used by the Flask API.

Table 7: CIFAR-10 class labels

Index	Class	Index	Class
0	Airplane	5	Dog
1	Automobile	6	Frog
2	Bird	7	Horse
3	Cat	8	Ship
4	Deer	9	Truck

3.5 Scope of This Report

The remainder of the report follows the sequence of the implementation. Section 5 describes the architecture, Sections 6 to 8 explain the environment, cluster and image preparation, Sections 9 and 10 explain the YAML manifests, Sections 11 to 13 cover deployment and verification of Pods and Services, Sections 14 and 15 document API testing and prediction output, Section 16 covers monitoring, and Sections 17 to 19 present results, observations and conclusions. Section 20 contains the complete screenshot set and Section 21 is an appendix with the full source code and configuration files.

4. System Architecture

4.1 Architecture Diagram

Figure 1 shows the architecture of the deployed system. The diagram groups the components into the Kubernetes deployment layer, the Streamlit frontend and the prediction backend.

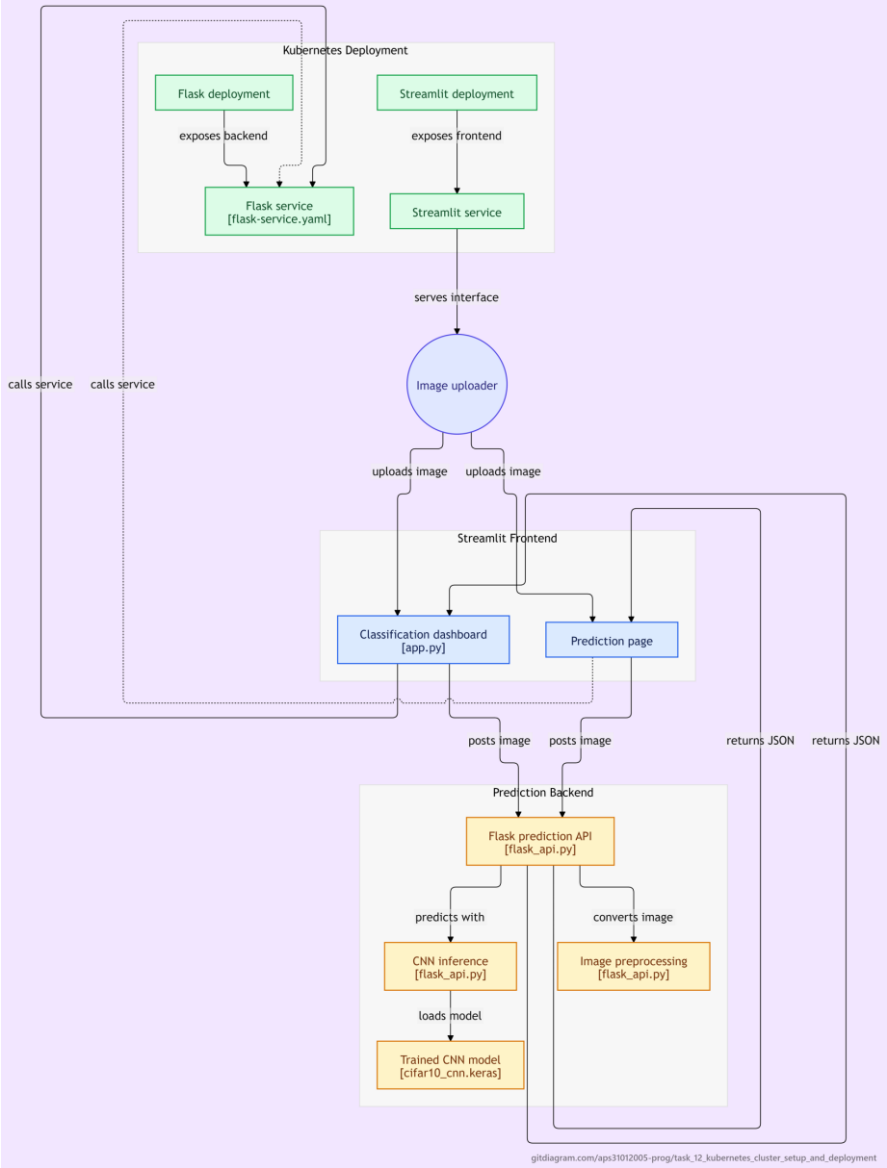


Figure 1: System architecture of the Kubernetes deployment

4.2 Reading the Diagram

The diagram has three main regions, described from top to bottom.

Kubernetes Deployment layer

At the top are the Flask deployment and the Streamlit deployment. The Flask deployment exposes the backend through the Flask service (flask-service.yaml), while the Streamlit deployment exposes the frontend through the Streamlit service. The Streamlit service serves the user interface to the person using the application, represented in the diagram as the image uploader.

Streamlit Frontend

The frontend has two entry points: the classification dashboard defined in app.py and the prediction page defined in pages/1_Image_Prediction.py. Both accept an uploaded image from the user. Both post the image to the Flask prediction API and both receive a JSON response. The dotted and solid lines labelled "calls service" show that the frontend reaches the backend through the Flask Kubernetes Service rather than directly through a Pod.

Prediction Backend

The backend contains the Flask prediction API (flask_api.py), which performs image preprocessing and CNN inference. The inference step loads the trained model file cifar10_cnn.keras. The result is returned to the frontend as JSON containing the predicted class and confidence.

4.3 Component Responsibilities

Table 8: Responsibilities of each architectural component

Component	Type	Responsibility
flask-api	Deployment	Keeps one replica of the Flask API Pod running
flask-api-service	Service (ClusterIP)	Stable internal address (10.109.95.216:5000) for the Flask Pods
streamlit	Deployment	Keeps one replica of the Streamlit Pod running
streamlit-service	Service (NodePort)	Exposes Streamlit outside the cluster (8501:31883)
flask_api.py	Application code	REST API with / and /predict endpoints
app.py	Application code	Dashboard, backend status check and image prediction UI
cifar10_cnn.keras	Model file	Trained CNN loaded from model/ at API start-up

4.4 Request Flow

A single prediction travels through the system as follows:

1. The user opens the Streamlit application through the NodePort service and uploads a JPG, JPEG or PNG image.
2. Streamlit displays the image and, when the Predict button is pressed, sends an HTTP POST with the file to `http://flask-api-service:5000/predict`.
3. Kubernetes DNS resolves `flask-api-service` to the Service ClusterIP, and kube-proxy forwards the request to the Flask Pod on port 5000.
4. The Flask API opens the file with Pillow, converts it to RGB, resizes it to 32 x 32 pixels, converts it to a NumPy array, scales values to the range 0 to 1 and adds a batch dimension.
5. The CNN model runs `model.predict` and the API selects the class with the highest probability using `argmax`.
6. The API returns JSON with `predicted_class` and `confidence` (as a percentage rounded to two decimals).

7. Streamlit displays the class, the confidence value, a progress bar and the raw JSON response.

4.5 Network Design

Two Service types were deliberately chosen to reflect who needs to reach each component. The Flask API is an internal backend; only the Streamlit Pod needs to call it, so a ClusterIP Service is sufficient and keeps the API unreachable from outside the cluster. The Streamlit frontend must be reachable by a person using a browser, so it is exposed through a NodePort Service. The assigned node port was 31883 and the Minikube service helper created a local tunnel URL for browser access.

Table 9: Services observed in the cluster

Service	Type	Cluster IP	Port(s)	Reachable From
flask-api-service	ClusterIP	10.109.95.216	5000/TCP	Inside the cluster only
streamlit-service	NodePort	10.106.207.53	8501:31883/TCP	Outside via node port / minikube service URL
kubernetes	ClusterIP	10.96.0.1	443/TCP	Default API server service

4.6 Design Decisions

- **Two separate Deployments** allow the frontend and the backend to be updated, restarted and scaled independently.
- **Service names instead of Pod IPs** make the connection stable, because Pod IP addresses change whenever a Pod is recreated.
- **imagePullPolicy: Never** forces Kubernetes to use the images already present in Minikube, avoiding the need for an external registry.
- **Model bundled into the Flask image** (`COPY model ./model`) keeps the API self-contained and simple for a learning environment.

5. Environment Setup

5.1 Overview

Before creating the cluster, a working local environment is required. The project was implemented on a Windows machine using Docker Desktop, Minikube, kubectl, Python and Visual Studio Code, and all commands were run in Windows PowerShell from the project folder `D:\Task12_Kubernetes`.

5.2 Prerequisites

Table 10: Environment prerequisites

Prerequisite	Purpose	Verification Command
Docker Desktop	Container runtime; hosts the Minikube node	docker --version
Minikube	Creates the local Kubernetes cluster	minikube version
kubectl	Communicates with the cluster	kubectl version --client
Python 3.11 (inside images)	Runtime for Flask and Streamlit	Defined by python:3.11-slim base image
Visual Studio Code	Editing code and YAML	Not required for cluster operation

The verification commands listed above are the standard way to confirm that each tool is installed and reachable from the PowerShell PATH. Docker Desktop must be running before Minikube is started, because the Docker driver creates the Kubernetes node as a container.

5.3 Project Folder Structure

All files of the task are kept in a single project folder named `Task12_Kubernetes`. The structure below lists every file used in the implementation.

Project structure

```
Task12_Kubernetes/
|
|-- model/
|   `-- cifar10_cnn.keras
|
|-- pages/
|   `-- 1_Image_Prediction.py
|
|-- screenshots/
|   |-- 01_minikube_cluster_verification.png
|   |-- 02_minikube_docker_images.png
|   |-- 03_flask_pod_running.png
|   |-- 04_kubernetes_pods_running.png
|   |-- 05_kubernetes_services.png
|   |-- 06_kubernetes_streamlit_application.png
|   |-- 07_kubernetes_flask_api_health.png
|   |-- 08_kubernetes_frog_prediction.png
|   |-- 09_kubernetes_truck_prediction.png
|   |-- 10_kubernetes_deployments.png
|   |-- 11_kubernetes_cluster_resources.png
|   |-- 12_streamlit_flask_api_connected.png
|   |-- 13_kubernetes_streamlit_frog_prediction.png
|   `-- 14_final_kubernetes_verification.png
|
|-- app.py
```

```
|-- flask_api.py
|-- frog.png
|-- truck.png
|
|-- Dockerfile.flask
|-- Dockerfile.streamlit
|-- docker-compose.yml
|-- .dockerignore
|-- requirements.txt
|
|-- flask-deployment.yaml
|-- flask-service.yaml
|-- streamlit-deployment.yaml
`-- streamlit-service.yaml
```

5.4 Purpose of Each File Group

Table 11: Project files grouped by purpose

File(s)	Group	Purpose
flask_api.py, model/cifar10_cnn.keras	Backend	REST API and trained model
app.py, pages/1_Image_Prediction.py	Frontend	Streamlit dashboard and prediction page
Dockerfile.flask, Dockerfile.streamlit, .dockerignore	Containerization	Instructions for building the two images
docker-compose.yml	Containerization (previous task)	Runs both containers on one Docker host
requirements.txt	Dependencies	Python packages installed in both images
flask-*.yaml, streamlit-*.yaml	Kubernetes	Deployment and Service manifests
frog.png, truck.png	Test data	Sample images used for API and UI testing
screenshots/	Documentation	Evidence of the deployment

5.5 Python Dependencies

Both images install the same requirements file. This is simple, although it means each image contains some packages it does not strictly need (for example, the Streamlit image also receives TensorFlow). The resulting image sizes of roughly 800 MB reflect this, as shown in Section 8.

requirements.txt

```
flask
streamlit
requests
tensorflow
numpy
pillow
```

6.6 Test Images

Two sample images, frog.png and truck.png, were used for testing throughout the task (shown in Section 14). Both belong to CIFAR-10 classes. They are much larger than 32 x 32 pixels, so the Flask API resizes them before inference.

6. Minikube Cluster Setup

6.1 Starting the Cluster

The local cluster was created with a single command that selects Docker as the driver:

```
minikube start --driver=docker
```

During start-up Minikube performs a sequence of operations: it creates a Docker container that acts as the Kubernetes node, prepares the container runtime inside it, pulls and starts the control plane components (API server, etcd, controller manager, scheduler), starts the kubelet and kube-proxy, configures a network plug-in, enables the default storage provisioner and writes connection details into the kubeconfig file so that kubectl can reach the cluster.

6.2 Verifying Cluster Status

Once start-up completes, the cluster status is checked with `minikube status`, and the Kubernetes node is checked with `kubectl get nodes`:

```
minikube status
kubectl get nodes
```

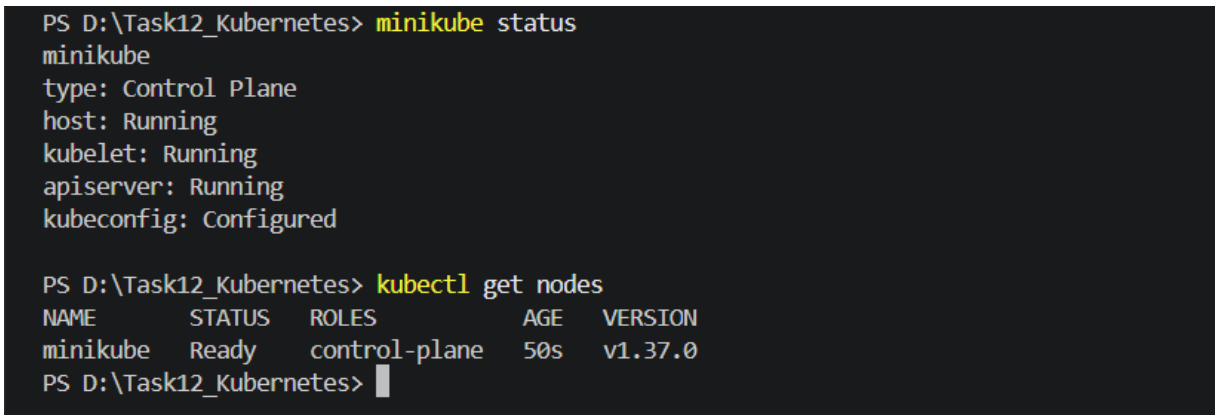


Figure 2: minikube status and kubectl get nodes output

6.3 Interpreting the Output

The output in the figure above confirms that the cluster is healthy. Each line has a specific meaning:

Table 12: Explanation of the status output

Output Line	Observed Value	Meaning
type	Control Plane	The Minikube node runs the Kubernetes control plane
host	Running	The Docker container hosting the node is running
kubelet	Running	The node agent that manages Pods is active
apiserver	Running	The Kubernetes API server accepts requests
kubeconfig	Configured	kubectl is configured to talk to this cluster
NAME	minikube	Name of the single node

Output Line	Observed Value	Meaning
STATUS	Ready	Node is healthy and can accept Pods
ROLES	control-plane	The same node runs control plane components
AGE	50s	Time since the node joined the cluster
VERSION	v1.37.0	Kubernetes version running on the node

6.4 Why the Node is Both Control Plane and Worker

In a production cluster the control plane is normally separated from the worker nodes. Minikube creates a single node for simplicity, and therefore this node must run application Pods as well. The Pods created later in this report (flask-api and streamlit) are scheduled onto this same node, which is why no additional nodes or node configuration were needed.

6.5 Kubernetes System Components Inside the Cluster

The images listed by Minikube (shown in Section 8) reveal the system components that make up the cluster. Each one has a specific role.

Table 13: Cluster system components

Component Image	Version	Role
kube-apiserver	v1.37.0	Front end of the control plane; every kubectl command goes through it
kube-controller-manager	v1.37.0	Runs controllers such as the Deployment and ReplicaSet controllers
kube-scheduler	v1.37.0	Decides which node a new Pod should run on
kube-proxy	v1.37.0	Implements Service networking rules on the node
etcd	3.7.0-0	Key-value store holding all cluster state
coredns	v1.14.6	Cluster DNS; resolves service names such as flask-api-service
pause	3.10.2	Infrastructure container that holds each Pod's network namespace
storage-provisioner	v5	Minikube default dynamic storage provisioner
kindnetd	v20260820 and v20250512	Network plug-in (CNI) used by Minikube for Pod networking

6.6 Why Cluster Verification Matters

Verifying the cluster before deploying any workload avoids a common source of confusion: if the node is not Ready, Pods remain in the Pending state and the problem can be mistaken for a mistake in the manifests. A node in the Ready state, together with an apiserver in the Running state, confirms that any later failure is caused by the application configuration and not by the cluster itself.

> **Result of this step:** The Minikube cluster was created successfully. The node `minikube` reached the Ready state with the control-plane role on Kubernetes version v1.37.0.

7. Docker Image

7.1 Why Images Are Needed

Kubernetes does not run source code; it runs container images. Each Pod created by a Deployment references an image, which contains the operating system layer, the Python runtime, the installed libraries and the application code. Two images were therefore required: one for the Flask API and one for the Streamlit frontend.

7.2 Dockerfile for the Flask API

Dockerfile.flask

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY flask_api.py .
COPY model ./model

EXPOSE 5000

CMD ["python", "flask_api.py"]
```

Table 14: Flask Dockerfile instructions

Instruction	Explanation
FROM python:3.11-slim	Starts from a small official Python 3.11 image
WORKDIR /app	Sets /app as the working directory inside the container
COPY requirements.txt .	Copies the dependency list first so that the install layer is cached
RUN pip install --no-cache-dir -r requirements.txt	Installs Flask, TensorFlow, NumPy, Pillow and other packages without keeping the pip cache
COPY flask_api.py .	Copies the API source code
COPY model ./model	Copies the trained model so the path model/cifar10_cnn.keras used by the code exists
EXPOSE 5000	Documents that the application listens on port 5000
CMD ["python", "flask_api.py"]	Starts the API when the container starts

7.3 Dockerfile for the Streamlit Frontend

Dockerfile.streamlit

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY pages ./pages

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

The Streamlit Dockerfile follows the same pattern. The two important differences are that it copies `app.py` and the `pages` folder instead of the API and the model, and that the start command runs Streamlit bound to `0.0.0.0` on port 8501. Binding to all interfaces is essential in a container: if Streamlit listened only on localhost, the Service could not forward traffic to it.

7.4 The .dockerignore File

.dockerignore

```
__pycache__
*.pyc
*.pyo
*.pyd
.git
.gitignore
.vscode
screenshots
screenshots9
README.md
```

The .dockerignore file keeps unnecessary files out of the build context. Python bytecode, version control data, editor settings, screenshots and the README are excluded. A smaller build context shortens the build and prevents accidental inclusion of unrelated files in the image.

7.5 Building Images Inside Minikube

The images were built with the `minikube image build` command. This builds the image using the container engine inside the Minikube node, so that the image is immediately available to Kubernetes without pushing it to a registry or loading it separately.

Image build commands

```
minikube image build -t task12-flask-api:latest -f Dockerfile.flask .

minikube image build -t task12-streamlit:latest -f Dockerfile.streamlit .
```

Table 15: Options used in minikube image build

Option	Meaning
-t task12-flask-api:latest	Tags the image with a name and the latest tag
-f Dockerfile.flask	Selects which Dockerfile to use (two Dockerfiles exist in the folder)
. (final argument)	Build context: the current project folder, filtered by .dockerignore

7.6 Verifying the Images

```
minikube image ls --format table
```

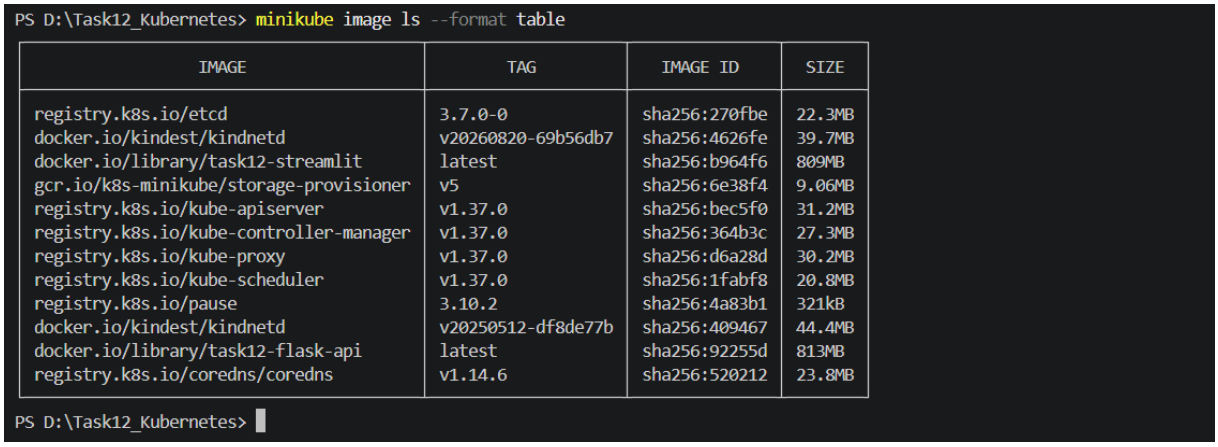


Figure 3: minikube image ls --format table output

The table lists all images known to the Minikube node. Alongside the Kubernetes system images, the two application images appear:

Table 16: Application images present in Minikube

Image	Tag	Image ID	Size
docker.io/library/task12-streamlit	latest	sha256:b964f6	809MB
docker.io/library/task12-flask-api	latest	sha256:92255d	813MB

Both images are large (about 800 MB) because TensorFlow is a heavy dependency and the same requirements.txt is used for both images. The next paragraph explains how Kubernetes is told to use these local images.

7.7 Why imagePullPolicy: Never Is Used

By default, Kubernetes tries to pull an image from a registry such as Docker Hub when a Pod starts, particularly for images tagged `latest`. The two application images exist only inside Minikube; they were never pushed to any registry. If Kubernetes attempted to pull them, the Pod would fail with `ErrImagePull` or `ImagePullBackOff`. Setting `imagePullPolicy: Never` in the Deployment tells the kubelet to use only the local image and never to contact a registry.

Table 17: imagePullPolicy values

imagePullPolicy	Behaviour	Suitable For
Always	Always pulls from the registry	Mutable tags in production registries
IfNotPresent	Pulls only if the image is not on the node	Versioned images from a registry
Never	Never pulls; fails if the image is missing locally	Local images built inside Minikube (used here)

> **Result of this step:** Both application images were built inside Minikube and verified with minikube image ls, so the Deployments can start Pods without an external registry.

8. Deployment YAML

8.1 What a Deployment Is

A Deployment is the standard Kubernetes object for running a stateless application. It declares the container image, the number of replicas and a Pod template. The Deployment controller creates a ReplicaSet, and the ReplicaSet creates and supervises the Pods. If a Pod fails or is deleted, the ReplicaSet creates a replacement so that the actual number of Pods always matches the declared number of replicas. Updating the Pod template causes the Deployment to create a new ReplicaSet and gradually move Pods from the old one to the new one.

8.2 Flask Deployment Manifest

flask-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-api
  template:
    metadata:
      labels:
        app: flask-api
    spec:
      containers:
        - name: flask-api
          image: task12-flask-api:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
```

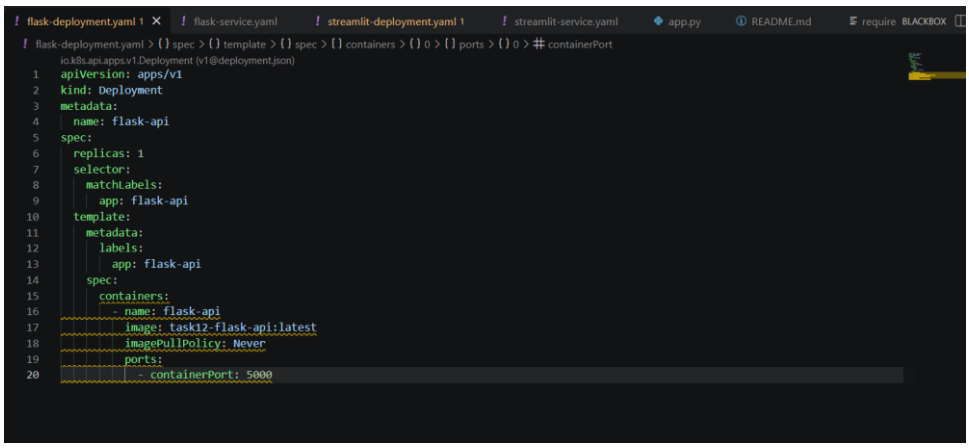


Figure 4: flask-deployment.yaml in the editor

8.3 Field-by-Field Explanation

Table 18: Fields of flask-deployment.yaml

Field	Value	Explanation
apiVersion	apps/v1	API group and version that defines the Deployment resource
kind	Deployment	Type of Kubernetes object being created
metadata.name	flask-api	Name of the Deployment; also used as the base of the Pod name
spec.replicas	1	Number of identical Pods to keep running
spec.selector.matchLabels	app: flask-api	Labels the Deployment uses to find the Pods it owns
template.metadata.labels	app: flask-api	Labels attached to every Pod; must match the selector
containers[0].name	flask-api	Name of the container inside the Pod
containers[0].image	task12-flask-api:latest	Image built inside Minikube
imagePullPolicy	Never	Use only the local image
containerPort	5000	Port on which the Flask application listens

8.4 Streamlit Deployment Manifest

streamlit-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: streamlit
spec:
  replicas: 1
  selector:
    matchLabels:
      app: streamlit
  template:
    metadata:
      labels:
        app: streamlit
    spec:
      containers:
        - name: streamlit
          image: task12-streamlit:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8501
```

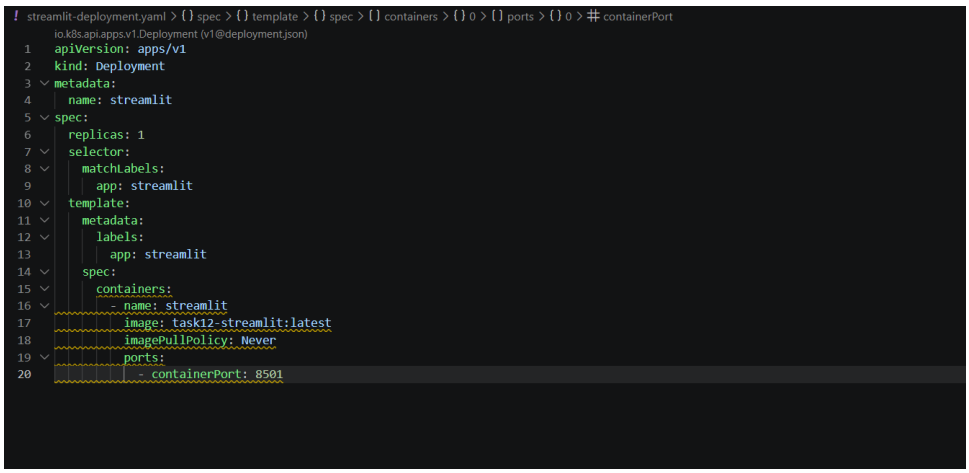


Figure 5: streamlit-deployment.yaml in the editor

The Streamlit Deployment has the same structure. Only the names, labels, image and container port differ. The label `app: streamlit` distinguishes its Pods from the Flask Pods, which is what allows each Service to select only its own Deployment's Pods.

8.5 Comparison of the Two Deployments

Table 19: Side-by-side comparison of Deployments

Property	flask-api	streamlit
Deployment name	flask-api	streamlit
Replicas	1	1
Pod label	app: flask-api	app: streamlit
Container name	flask-api	streamlit
Image	task12-flask-api:latest	task12-streamlit:latest
Image pull policy	Never	Never
Container port	5000	8501

8.6 The Label and Selector Contract

The most common source of YAML configuration errors is a mismatch between labels and selectors. Three places must agree for a Deployment to work: the `spec.selector.matchLabels` of the Deployment, the `template.metadata.labels` of the Pod template, and, later, the `spec.selector` of the Service. In both manifests of this project the values match exactly (`app: flask-api` and `app: streamlit`). If they did not, the Deployment would be rejected by the API server or the Service would have no endpoints.

8.7 Applying the Deployments

```
kubectl apply -f flask-deployment.yaml

kubectl apply -f streamlit-deployment.yaml
```

`kubectl apply` submits the manifest to the API server. The command is declarative and idempotent: running it again with an unchanged file changes nothing, and running it with a modified file updates the existing object. This is preferable to `kubectl create`, which fails if the object already exists.

> **Result of this step:** Two Deployment manifests were written with consistent names, labels, images and ports, and were applied to the cluster.

9. Service YAML

9.1 Why Services Are Needed

A Pod receives its own IP address, but that address is temporary: when a Pod is deleted or replaced, the new Pod gets a different address. Clients therefore cannot depend on Pod IP addresses. A Service solves this by providing a stable virtual IP address (the ClusterIP) and a DNS name. The Service selects Pods by label, and kube-proxy forwards traffic sent to the Service to one of the matching Pods. Even if Pods are recreated, the Service address remains the same.

9.2 Flask Service Manifest

flask-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
spec:
  selector:
    app: flask-api
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: ClusterIP
```

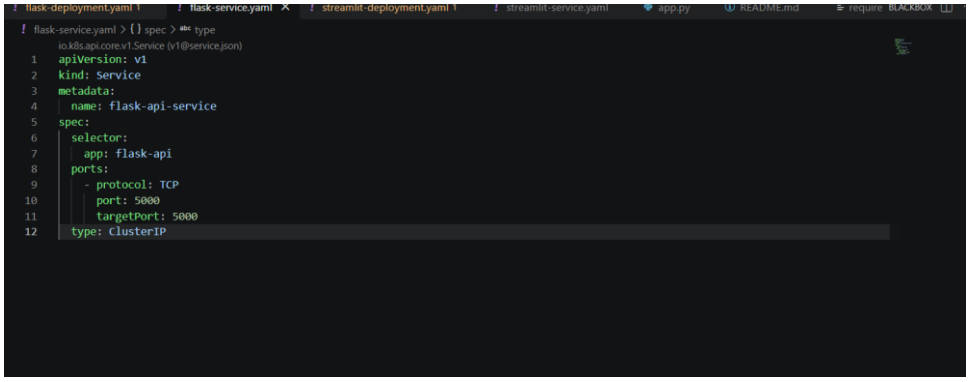


Figure 6: flask-service.yaml in the editor

Table 20: Fields of flask-service.yaml

Field	Value	Explanation
kind	Service	Creates a stable network endpoint
metadata.name	flask-api-service	Becomes the cluster DNS name used by Streamlit
spec.selector	app: flask-api	Forwards traffic to Pods with this label
port	5000	Port exposed by the Service
targetPort	5000	Container port that receives the traffic
protocol	TCP	Transport protocol
type	ClusterIP	Reachable only from inside the cluster

9.3 Streamlit Service Manifest

streamlit-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: streamlit-service
spec:
  selector:
    app: streamlit
  ports:
    - protocol: TCP
      port: 8501
      targetPort: 8501
  type: NodePort
```

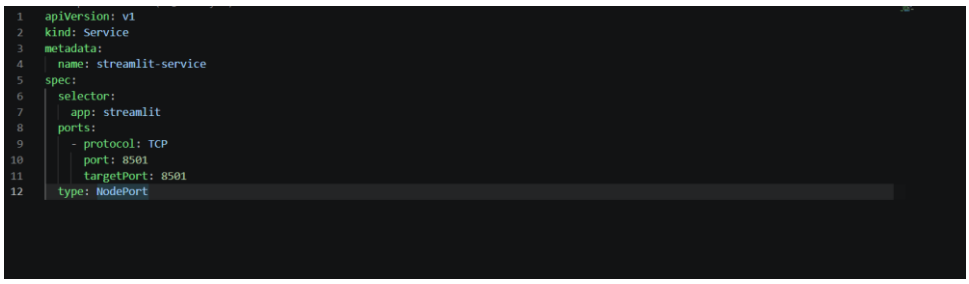


Figure 7: streamlit-service.yaml in the editor

The Streamlit Service differs in two important ways. Its selector targets Pods labelled `app: streamlit`, and its type is `NodePort`. A NodePort Service opens a port on the node (in the range 30000 to 32767) and forwards traffic from that port to the Service and then to the Pod. No node port was specified in the manifest, so Kubernetes assigned one automatically; `kubectl get services` later showed it as 31883.

9.4 ClusterIP versus NodePort

Table 21: Service types used in the project

Aspect	ClusterIP (Flask)	NodePort (Streamlit)
Accessible from	Inside the cluster only	Inside the cluster and from outside through the node
Gets a ClusterIP	Yes	Yes
Opens a port on the node	No	Yes (31883 in this deployment)
Typical use	Internal backends and databases	Development exposure of frontends
Reason for choice here	Only Streamlit needs the API	A user must open the UI in a browser

9.5 Service Discovery Through DNS

CoreDNS creates a DNS record for every Service. A Pod in the same namespace can therefore use the Service name as a host name. The line in app.py `FLASK_API_URL = "http://flask-api-service:5000"` depends on this mechanism: the name `flask-api-service` is resolved to the ClusterIP 10.109.95.216 and the request is forwarded to the Flask Pod. The name in the Python code must match `metadata.name` in flask-service.yaml exactly, and it does.

> **Result of this step:** A ClusterIP Service for the Flask API and a NodePort Service for the Streamlit frontend were defined with selectors matching their Deployments.

10. Kubernetes Deployment

10.1 Deployment Sequence

With the cluster running, the images built and the manifests prepared, the application was deployed by applying the four YAML files. The backend was deployed first, followed by the frontend, because the frontend depends on the backend service name.

Commands used for deployment

```
kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
kubectl apply -f streamlit-deployment.yaml
kubectl apply -f streamlit-service.yaml
```

Table 22: Deployment steps

Step	Command	Object Created
1	kubectl apply -f flask-deployment.yaml	Deployment flask-api (and its ReplicaSet and Pod)
2	kubectl apply -f flask-service.yaml	Service flask-api-service (ClusterIP)
3	kubectl apply -f streamlit-deployment.yaml	Deployment streamlit (and its ReplicaSet and Pod)
4	kubectl apply -f streamlit-service.yaml	Service streamlit-service (NodePort)

10.2 What Happens After kubectl apply

A successful `kubectl apply` returns immediately, but a chain of events then takes place inside the cluster:

1. kubectl sends the manifest to the API server, which validates it and stores the Deployment object in etcd.
2. The Deployment controller notices the new object and creates a ReplicaSet with the Pod template.
3. The ReplicaSet controller notices that zero Pods exist while one is desired, and creates a Pod.
4. The scheduler assigns the unscheduled Pod to a node; in this cluster there is only the node `minikube`.
5. The kubelet on the node sees the assignment and asks the container runtime to start the container from the image (no pull occurs because of imagePullPolicy: Never).
6. The container starts; the Pod phase changes from Pending to Running and the READY column shows 1/1.
7. For a Service, the endpoints controller records the IP addresses of the Pods that match the selector, and kube-proxy programs the forwarding rules.

10.3 Start-Up Behaviour of the Flask Pod

The Flask container executes `python flask_api.py`. At start-up the module loads the TensorFlow library and then calls `tf.keras.models.load_model("model/cifar10_cnn.keras")`. Because the model is loaded at the module level, it is read once when the process starts and is then kept in memory for all subsequent requests. Loading is the slowest part of start-up, which explains why the first checks of the Flask Pod should be made only after its status shows Running.

10.4 Deployment Timeline

The ages displayed by kubectl in the screenshots allow the order of events to be reconstructed. The Flask Pod was the oldest resource, the Flask Service followed, and the Streamlit resources were created last.

Table 23: Ages reported by kubectl

Observation	Value	Source Screenshot
Flask Pod age at first check	44s (Running, 1/1, 0 restarts)	Flask pod running
Flask Pod age when Streamlit Pod appeared	2m24s	Kubernetes pods running
Streamlit Pod age at that time	43s (Running, 1/1, 0 restarts)	Kubernetes pods running
flask-api-service age	3m11s	Kubernetes services
streamlit-service age	90s	Kubernetes services
kubernetes default service age	53m	Kubernetes services

10.5 Accessing the Streamlit Application

Because Streamlit is exposed as a NodePort Service inside a Docker-driver Minikube cluster on Windows, the node port is not directly reachable from the host browser. Minikube therefore provides a helper that creates a local tunnel and prints a URL:

```
minikube service streamlit-service --url
```

The resulting address in the browser was `127.0.0.1:64394`. The screenshot below shows the Streamlit dashboard served from the Pod running inside Kubernetes, with the key dataset metrics, the ten class categories and the sidebar showing the Image Prediction page.

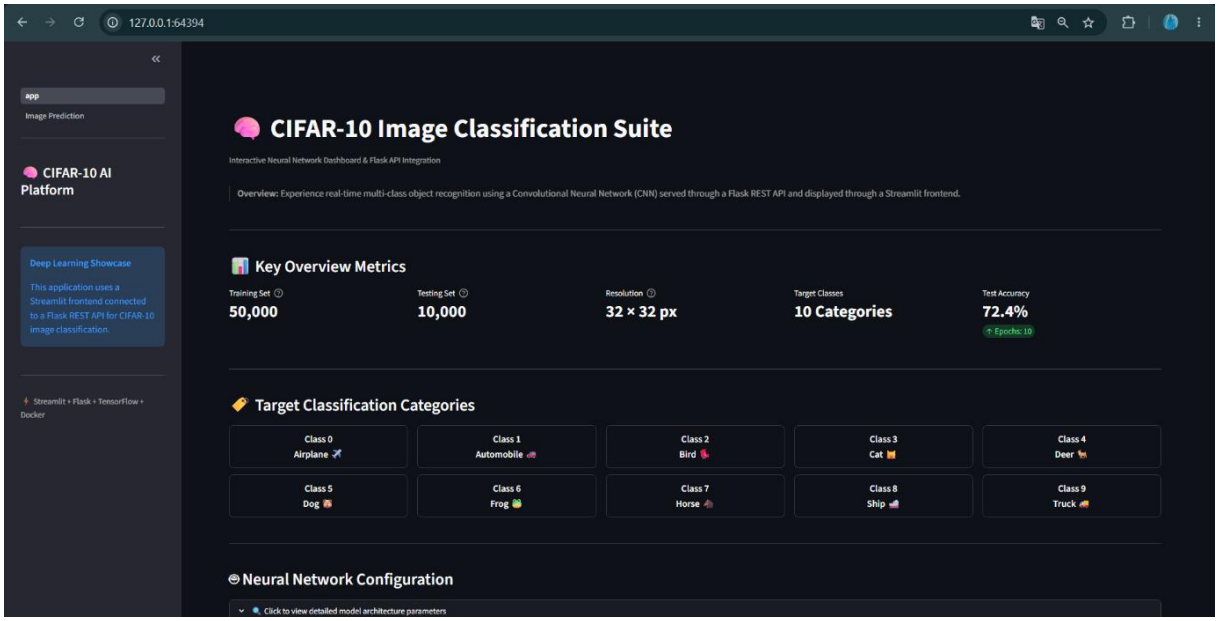


Figure 8: Streamlit application running in Kubernetes

10.6 What the Dashboard Shows

Table 24: Content of the Streamlit dashboard

Dashboard Element	Displayed Value
Title	CIFAR-10 Image Classification Suite
Training Set	50,000 images
Testing Set	10,000 images
Resolution	32 x 32 px
Target Classes	10 Categories
Test Accuracy	72.4% (Epochs: 10)
Sidebar pages	app and Image Prediction

> **Result of this step:** All four manifests were applied. Both applications started inside the cluster and the Streamlit interface opened in the browser through the Minikube service URL.

11. Pod Verification

11.1 Understanding Pods

A Pod is the smallest unit that Kubernetes schedules. It wraps one or more containers that share a network namespace (and therefore an IP address and ports) and optional storage volumes. In this project each Pod contains exactly one container. Pods are not created by hand; they are created and owned by a ReplicaSet which is in turn owned by a Deployment.

11.2 Verifying the Flask Pod

After applying the Flask Deployment, the Pod was checked with:

```
kubectl get pods
```

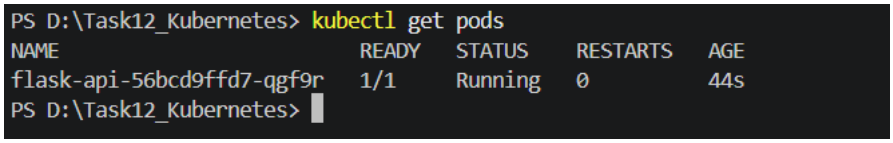


Figure 9: Flask Pod in Running state

11.3 Verifying Both Pods

After the Streamlit Deployment was applied, the same command listed both Pods:

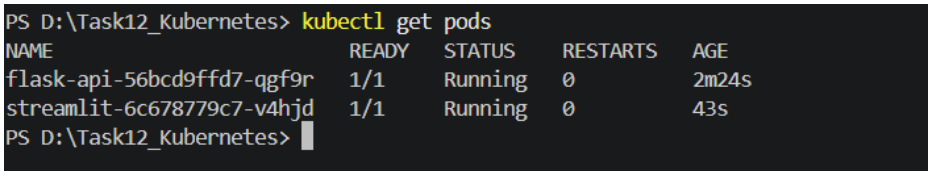


Figure 10: Flask and Streamlit Pods both in Running state

11.4 Reading the Pod Table

Table 25: Pod status columns

Column	Flask Pod Value	Streamlit Pod Value	Meaning
NAME	flask-api-56bcd9ffd7-qgf9r	streamlit-6c678779c7-v4hjd	Deployment name, ReplicaSet hash and Pod suffix
READY	1/1	1/1	One of one containers is ready
STATUS	Running	Running	Pod is scheduled and its container is running
RESTARTS	0	0	The container has never crashed or restarted
AGE	2m24s	43s	Time since Pod creation

11.5 Anatomy of a Pod Name

Kubernetes generates Pod names automatically. The name `flask-api-56bcd9ffd7-qgf9r` has three parts: `flask-api` is the name of the owning Deployment, `56bcd9ffd7` is a hash of the Pod template that identifies the ReplicaSet, and `qgf9r` is a random suffix that makes the Pod name unique. When the Pod template changes (for example after a rollout restart), the hash changes, which is why the Streamlit Pod later appears as `streamlit-c8df87dbd-pw8s8`.

11.6 Pod Lifecycle Phases

Table 26: Common Pod states

Phase / State	Description
Pending	The Pod is accepted but its containers are not yet running (scheduling or image preparation)
ContainerCreating	The node is setting up the container
Running	At least one container is running; this is the state observed for both Pods
Succeeded / Failed	All containers ended successfully / at least one ended in error (not applicable to long-running services)
CrashLoopBackOff	The container repeatedly crashes and Kubernetes waits longer between restarts
ImagePullBackOff / ErrImagePull	The image could not be obtained; a typical symptom of a wrong image name or missing local image

11.7 Why 0 Restarts Is Significant

A Pod can show the Running status and still be unhealthy if its container keeps restarting. The RESTARTS value of 0 for both Pods shows that neither container crashed after start. For the Flask Pod this means that the model loaded without error and that the Python process has remained stable. For the Streamlit Pod it means that the application started and continued to serve requests.

11.8 Troubleshooting Reference

No failures occurred in this deployment, so no troubleshooting was required. For completeness, the table below lists the checks that would be applied if a Pod had not reached the Running state.

Table 27: Pod troubleshooting reference

Symptom	Likely Cause	Check
ImagePullBackOff	Image missing in Minikube or wrong image name	minikube image ls; compare with image: in the manifest
CrashLoopBackOff	Application error at start-up, such as a missing model file	kubectl logs <pod>
Pending for a long time	Node not Ready or insufficient resources	kubectl get nodes; kubectl describe pod <pod>
Running but not serving	Application bound to 127.0.0.1 instead of 0.0.0.0	Check the start command and host setting

> **Result of this step:** Both Pods reached the Running state with 1/1 containers ready and 0 restarts.

12. Service Verification

12.1 Listing the Services

```
kubectl get services
```

```
PS D:\Task12_Kubernetes> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
flask-api-service   ClusterIP   10.109.95.216   <none>        5000/TCP         3m11s
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          53m
streamlit-service   NodePort    10.106.207.53   <none>        8501:31883/TCP   90s
PS D:\Task12_Kubernetes>
```

Figure 11: kubectl get services output

12.2 Reading the Service Table

Table 28: Services in the cluster

Service	Type	Cluster IP	External IP	Ports	Age
flask-api-service	ClusterIP	10.109.95.216	<none>	5000/TCP	3m11s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	53m
streamlit-service	NodePort	10.106.207.53	<none>	8501:31883/TCP	90s

Three Services exist. The `kubernetes` Service is created automatically by Kubernetes and gives Pods access to the API server. The other two are the application Services defined in this project.

The Port(s) Column

For `flask-api-service` the value `5000/TCP` means the Service listens on port 5000 and forwards to target port 5000. For `streamlit-service` the value `8501:31883/TCP` has the format `port:nodePort`. Port 8501 is the Service port inside the cluster and 31883 is the port opened on the node. The node port was assigned automatically because the manifest did not specify one.

The External IP Column

All services show <none> for EXTERNAL-IP. This is expected: external IP addresses are allocated for LoadBalancer Services, which this project does not use. NodePort access relies on the node address and the node port rather than on an external IP.

12.3 Verifying the Backend Service

Because flask-api-service is of type ClusterIP, it cannot be opened directly from a browser on the host. To test the backend manually, the Service was temporarily exposed to the local machine with port forwarding:

```
kubectl port-forward service/flask-api-service 5000:5000
```

This command opens a tunnel between local port 5000 and port 5000 of the Service, and stays in the foreground while the tunnel is active. It is a testing aid; it does not change the Service type and does not make the API publicly reachable. Results obtained through this tunnel are covered in Section 14.

12.4 Verifying Pod-to-Pod Communication

The most important Service check is whether the Streamlit Pod can reach the Flask Pod through the Service name. The Streamlit dashboard contains a Backend API Status section with a button that sends a GET request to `http://flask-api-service:5000/`. The screenshot below shows that the connection succeeded and that the Flask API returned its JSON status message.



Figure 12: Streamlit backend API status check succeeded

The success message confirms three things at once: CoreDNS resolved the Service name, the Service forwarded the request to the Flask Pod, and the Flask application responded with HTTP 200. The JSON shown in the figure contains the `endpoint` (`/predict`), the `message` (`CIFAR-10 Flask Prediction API is running`) and the `method` (`POST`).

12.5 Service Verification Summary

Table 29: Service verification summary

Check	Method	Result
Flask Service exists as ClusterIP	kubectl get services	Confirmed (10.109.95.216)
Streamlit Service exists as NodePort	kubectl get services	Confirmed (8501:31883/TCP)
Streamlit reachable from browser	minikube service streamlit-service --url	Dashboard opened
Flask reachable from local machine	kubectl port-forward	Health response returned
Streamlit reaches Flask inside cluster	Check Flask API Connection button	Connected and running

> **Result of this step:** Both Services were verified, and in-cluster communication between the Streamlit Pod and the Flask Pod through the Service name was confirmed.

13. API Testing

13.1 API Overview

The Flask application exposes two endpoints. The home endpoint is used as a health check and the prediction endpoint performs inference.

Table 30: Flask API endpoints

Endpoint	Method	Input	Output
/	GET	None	JSON message that the API is running, the endpoint name and the method
/predict	POST	Multipart form field named image	JSON with predicted_class and confidence

13.2 Health Check Test

With the port-forward tunnel active, the health endpoint was opened in a browser at `http://127.0.0.1:5000/`.

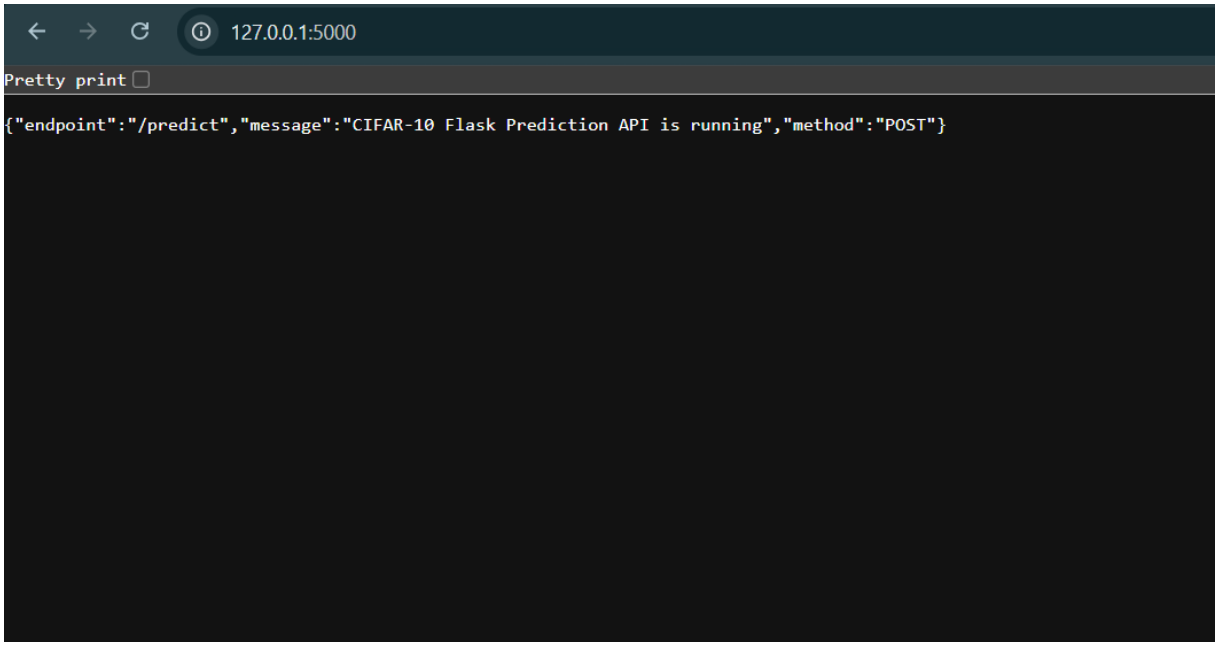


Figure 13: Flask health endpoint response through port-forward

The browser displayed a compact JSON document stating that the CIFAR-10 Flask Prediction API is running, together with the name of the prediction endpoint and its HTTP method. This confirms that the Flask Pod is healthy and that the Service and the tunnel forward requests correctly.

13.3 Prediction Endpoint Implementation

The `/predict` function first checks that the request contains a file named `image`. If it does not, the API returns HTTP 400 with an error message. Otherwise the image is preprocessed and passed to the model. Any unexpected exception is caught and returned as HTTP 500 with the exception text.

Core of flask_api.py (abridged; full file in Appendix)

```
@app.route("/predict", methods=["POST"])
def predict():
    if "image" not in request.files:
        return jsonify({"error": "No image provided. Please upload an image."}), 400
    try:
        image_file = request.files["image"]
        image = Image.open(image_file).convert("RGB")
        image = image.resize((32, 32))
        image_array = np.array(image)
        image_array = image_array.astype("float32") / 255.0
        image_array = np.expand_dims(image_array, axis=0)
        predictions = model.predict(image_array, verbose=0)
        predicted_index = np.argmax(predictions[0])
        predicted_class = class_names[predicted_index]
        confidence = float(predictions[0][predicted_index]) * 100
        return jsonify({"predicted_class": predicted_class,
                        "confidence": round(confidence, 2)})
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

13.4 Preprocessing Pipeline

Table 31: Image preprocessing and inference steps

Step	Operation	Reason
1	Open the uploaded file with Pillow and convert to RGB	Ensures exactly three channels, even for PNG files with transparency
2	Resize to 32 x 32 pixels	The CNN was trained on 32 x 32 CIFAR-10 images
3	Convert to a NumPy array	Model input must be numeric
4	Cast to float32 and divide by 255	Scale pixel values to the range 0 to 1, as in training
5	Add a batch dimension (shape 1 x 32 x 32 x 3)	Keras models expect a batch of images
6	model.predict and argmax	Select the class with the highest softmax probability
7	Multiply the probability by 100 and round to 2 decimals	Express the confidence as a percentage

13.5 Prediction Test with the Frog Image

The frog image was submitted to the Flask API through the port-forward tunnel using curl:

```
curl.exe -X POST -F "image=@frog.png" http://127.0.0.1:5000/predict
```



Figure 14: Input image: frog.png

```
PS D:\Task12_Kubernetes> cd D:\Task12_Kubernetes
PS D:\Task12_Kubernetes> curl.exe -X POST -F "image=@frog.png" http://127.0.0.1:5000/predict
{"confidence":99.01,"predicted_class":"Frog"}
PS D:\Task12_Kubernetes>
```

Figure 15: curl prediction request for the frog image

The API returned `{"confidence":99.01,"predicted_class":"Frog"}`. The model classified the image correctly with a confidence of 99.01 percent.

13.6 Prediction Test with the Truck Image

```
curl.exe -X POST -F "image=@truck.png" http://127.0.0.1:5000/predict
```



Figure 16: Input image: truck.png

```
PS D:\Task12_Kubernetes> cd D:\Task12_Kubernetes
PS D:\Task12_Kubernetes> curl.exe -X POST -F "image=@truck.png" http://127.0.0.1:5000/predict
{"confidence":86.98,"predicted_class":"Truck"}
PS D:\Task12_Kubernetes>
```

Figure 17: curl prediction request for the truck image

The API returned `{"confidence":86.98,"predicted_class":"Truck"}`. The truck was also classified correctly, with a lower confidence of 86.98 percent.

13.7 Explanation of the curl Command

Table 32: Breakdown of the curl request

Part	Meaning
curl.exe	Explicit .exe name is used in PowerShell, where curl is otherwise an alias for Invoke-WebRequest
-X POST	Sends an HTTP POST request
-F "image=@frog.png"	Sends a multipart/form-data field named image containing the contents of frog.png
http://127.0.0.1:5000/predict	The forwarded local address that reaches flask-api-service inside the cluster

> **Result of this step:** The Flask API running in Kubernetes returned correct predictions for both test images, proving that the containerized CNN model is accessible and functional.

14. Prediction Output

14.1 Output Format

Every successful prediction returns a JSON object with two fields. The field `predicted_class` holds the name of the class with the highest probability, and `confidence` holds that probability as a percentage with two decimals.

Example response

```
{
  "confidence": 99.01,
  "predicted_class": "Frog"
}
```

14.2 Results Through the Flask API

Table 33: Predictions returned by the Flask API

Test Image	Expected Class	Predicted Class	Confidence	Correct
frog.png	Frog	Frog	99.01%	Yes
truck.png	Truck	Truck	86.98%	Yes

14.3 End-to-End Prediction Through Streamlit

The complete workflow was also tested from the browser. The frog image was uploaded in the Image Prediction section of the Streamlit application and the Predict Image button was pressed. Streamlit posted the image to `http://flask-api-service:5000/predict`, and displayed the answer.

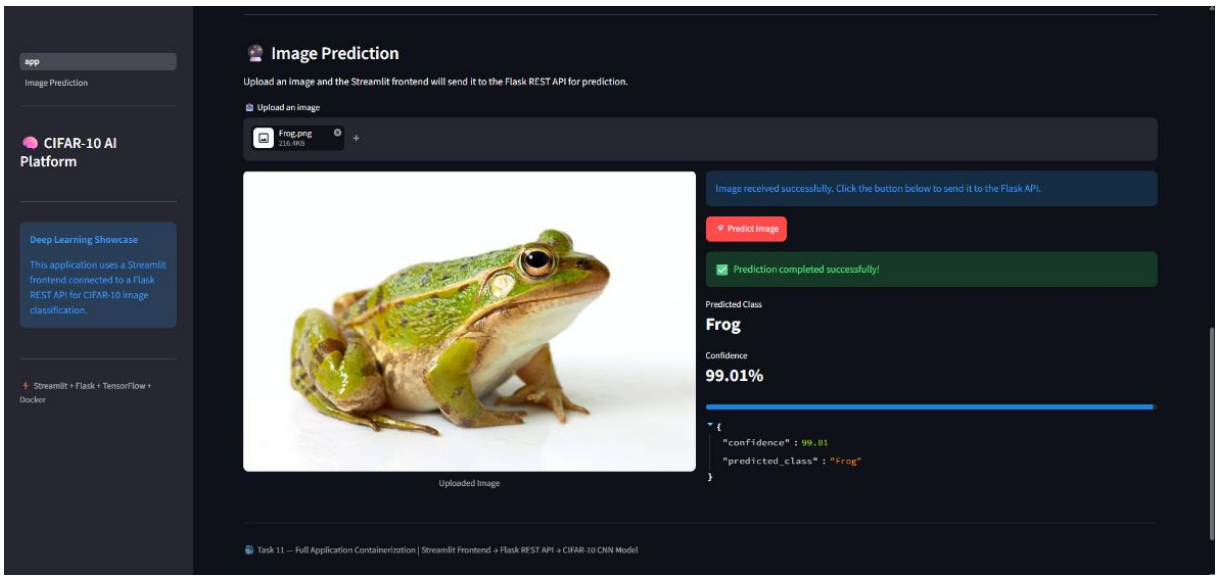


Figure 18: Prediction made through the Streamlit application

The interface displayed a success message, the predicted class Frog, the confidence 99.01 percent, a progress bar filled to the same level and the raw JSON returned by the API. The result is identical to the one obtained with curl, which shows that the same backend serves both clients consistently.

14.4 Elements of the Streamlit Result

Table 34: Elements of the Streamlit prediction result

Element	Displayed Value	Source
Uploaded file	Frog.png (216.4KB)	st.file_uploader
Status message	Prediction completed successfully!	HTTP 200 from Flask
Predicted Class	Frog	predicted_class field
Confidence	99.01%	confidence field
Progress bar	Filled to 99.01%	confidence / 100
JSON panel	confidence 99.01, predicted_class Frog	Raw API response

14.5 Interpreting Confidence

The confidence value is the softmax probability of the chosen class. A value of 99.01 percent indicates that the model is very sure about the frog image, whereas 86.98 percent for the truck indicates somewhat lower certainty. These values should be interpreted carefully. They describe how strongly the model prefers one class over the others for that particular input, but they are not a guarantee of correctness. The model has an overall test accuracy of 72.4 percent on the CIFAR-10 test set, so it can make mistakes, especially on images that differ from the training data.

14.6 Scope of the Testing

Only two sample images were used for the deployment tests. The purpose of the task is to validate the Kubernetes deployment and the integration of the components, not to re-evaluate the accuracy of the model. The two tests are sufficient for that purpose because they exercise the full path (upload, network, preprocessing, inference, response) and return correct, plausible results.

> **Result of this step:** Predictions through both the API and the Streamlit interface were successful and consistent.

15. Monitoring

15.1 Monitoring Approach

Monitoring in this task was performed with kubectl status commands. They give an immediate view of the desired and the actual state of each resource. The following commands were used.

Table 35: Monitoring commands used

Command	Purpose
kubectl get pods	List Pods with status, restarts and age
kubectl get services	List Services with type, IPs and ports
kubectl get deployments	List Deployments with replica counts
kubectl get pods,services,deployments	Combined view of the three resource types
kubectl get all	View of all common resources, including ReplicaSets
kubectl rollout restart deployment streamlit	Recreate the Streamlit Pods from the Deployment
kubectl rollout status deployment streamlit	Wait for and report the progress of a rollout

15.2 Deployment Status

```
kubectl get deployments
```

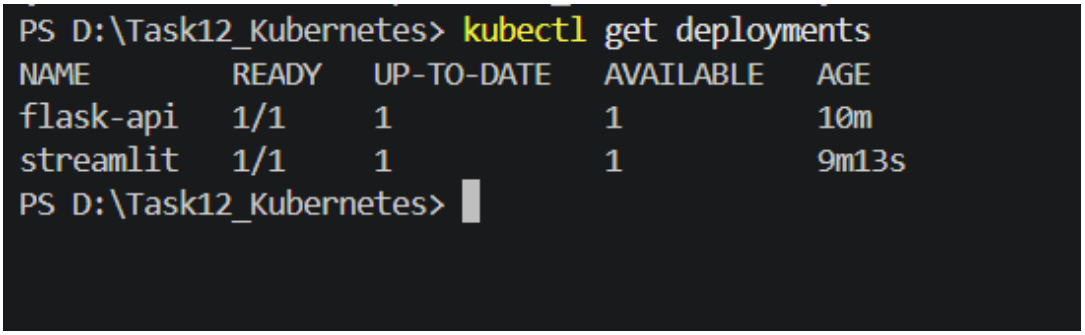


Figure 19: kubectl get deployments output

Table 36: Deployment status columns

Column	flask-api	streamlit	Meaning
READY	1/1	1/1	Ready Pods out of desired Pods
UP-TO-DATE	1	1	Pods updated to the latest Pod template
AVAILABLE	1	1	Pods available to serve traffic
AGE	10m	9m13s	Time since Deployment creation

15.3 Combined Resource View

```
kubectl get pods,services,deployments
```

```
PS D:\Task12_Kubernetes> kubectl get pods,services,deployments
NAME                                 READY   STATUS    RESTARTS   AGE
pod/flask-api-56bcd9ffd7-qgf9r       1/1     Running   0          11m
pod/streamlit-6c678779c7-v4hjd       1/1     Running   0          9m56s

NAME                         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/flask-api-service    ClusterIP   10.109.95.216    <none>        5000/TCP         11m
service/kubernetes           ClusterIP   10.96.0.1        <none>        443/TCP          62m
service/streamlit-service    NodePort    10.106.207.53    <none>        8501:31883/TCP   9m49s

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-api    1/1     1            1           11m
deployment.apps/streamlit    1/1     1            1           9m56s
PS D:\Task12_Kubernetes>
```

Figure 20: kubectl get pods,services,deployments output

The combined view shows Pods, Services and Deployments in one output. Both Pods are Running with 0 restarts. Both application Services have the expected types and ports. Both Deployments report 1/1 ready, 1 up to date and 1 available.

15.4 Final Cluster Verification

```
kubectl get all
```

```
PS D:\Task12_Kubernetes> kubectl get all
NAME                                 READY   STATUS    RESTARTS   AGE
pod/flask-api-56bcd9ffd7-qgf9r       1/1     Running   0          19m
pod/streamlit-c8df87dbd-pw8s8        1/1     Running   0          3m29s

NAME                         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/flask-api-service    ClusterIP   10.109.95.216    <none>        5000/TCP         19m
service/kubernetes           ClusterIP   10.96.0.1        <none>        443/TCP          70m
service/streamlit-service    NodePort    10.106.207.53    <none>        8501:31883/TCP   18m

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-api    1/1     1            1           19m
deployment.apps/streamlit    1/1     1            1           18m

NAME                                    DESIRED   CURRENT   READY   AGE
replicaset.apps/flask-api-56bcd9ffd7    1         1         1       19m
replicaset.apps/streamlit-6c678779c7    0         0         0       18m
replicaset.apps/streamlit-c8df87dbd     1         1         1       3m29s
PS D:\Task12_Kubernetes>
```

Figure 21: kubectl get all output (final verification)

The last verification uses `kubectl get all`, which additionally lists the ReplicaSets. The output contains useful information about the history of the Streamlit Deployment.

Table 37: ReplicaSets in the final verification

ReplicaSet	Desired	Current	Ready	Age	Interpretation
flask-api-56bcd9ffd7	1	1	1	19m	Active ReplicaSet of the Flask API
streamlit-6c678779c7	0	0	0	18m	Old ReplicaSet, scaled to zero after the restart
streamlit-c8df87dbd	1	1	1	3m29s	Current ReplicaSet with the running Streamlit Pod

15.5 Understanding the Rollout

Earlier the Streamlit Pod was named `streamlit-6c678779c7-v4hjd`. In the final screenshot the running Pod is `streamlit-c8df87dbd-pw8s8`, which belongs to a new ReplicaSet. This is the visible effect of `kubectl rollout restart deployment streamlit`. A rollout restart changes the Pod template annotations so that the Deployment controller creates a new ReplicaSet, starts the new Pod, and scales the old ReplicaSet down to zero. The old ReplicaSet is kept (with zero Pods) so that the Deployment can be rolled back if needed.

The Flask Deployment was not restarted, so its Pod `flask-api-56bcd9ffd7-qgf9r` has the same name in every screenshot, with an age that increases from 44s to 19m. This continuity also shows that the backend remained stable for the entire test period without restarts.

```
kubectl rollout restart deployment streamlit
kubectl rollout status deployment streamlit
```

15.6 Comparison Across Monitoring Snapshots

Table 38: Monitoring snapshots over time

Snapshot	Flask Pod	Streamlit Pod	Observation
Pods (first check)	Running, 44s	Not yet created	Backend deployed first
Pods (both)	Running, 2m24s	Running, 43s	Frontend deployed
Deployments	1/1 (10m)	1/1 (9m13s)	Both available
Pods, services, deployments	Running, 11m	Running, 9m56s	Resources consistent
kubectl get all	Running, 19m	Running, 3m29s (new Pod)	Rollout restart of Streamlit completed

15.7 Additional Commands for Deeper Monitoring

The following commands were not captured in the screenshots of this task, but they are the standard next tools for a deeper investigation of a deployment and are listed here as a reference.

Table 39: Reference commands for deeper monitoring

Command	Information Provided
kubectl describe pod <pod-name>	Events, container state, image, node and IP of a Pod
kubectl logs <pod-name>	Standard output of the container (for example Flask request logs)
kubectl get endpoints	Pod IP addresses behind each Service
kubectl get events	Recent cluster events, useful for scheduling and image problems
minikube dashboard	Web-based overview of cluster resources

> **Result of this step:** All resources were monitored through kubectl. Both Deployments showed 1/1 ready, both Pods showed Running with 0 restarts, and the ReplicaSet history correctly reflected the Streamlit rollout.

16. Results

16.1 Overall Result

The Kubernetes deployment of the CIFAR-10 application was completed successfully. All planned components were created, all verification commands returned the expected status, and the application produced correct predictions both through the REST API and through the graphical interface.

16.2 Verification Results

Table 40: Summary of verification results

Operation	Result	Evidence
Minikube cluster setup	Successful	minikube status: host, kubelet and apiserver Running
Kubernetes node verification	Ready	kubectl get nodes: minikube, Ready, v1.37.0
Flask Docker image	Available	task12-flask-api:latest (813MB)
Streamlit Docker image	Available	task12-streamlit:latest (809MB)
Flask Deployment	Running	1/1 ready, 1 up-to-date, 1 available
Streamlit Deployment	Running	1/1 ready, 1 up-to-date, 1 available
Flask Service	Available	ClusterIP 10.109.95.216, port 5000/TCP
Streamlit Service	Available	NodePort 10.106.207.53, port 8501:31883/TCP
Flask health check	Successful	JSON status message returned at /
Frog API prediction	Successful	Frog, 99.01%
Truck API prediction	Successful	Truck, 86.98%
Streamlit to Flask connection	Successful	Flask API is connected and running
Streamlit prediction	Successful	Frog, 99.01% displayed in the UI
Final Kubernetes resource verification	Successful	kubectl get all: all Pods Running, 0 restarts

16.3 Resource Inventory

Table 41: Final resource inventory

Resource Type	Name	Status
Node	minikube	Ready (control-plane)
Deployment	flask-api	1/1 available
Deployment	streamlit	1/1 available
ReplicaSet	flask-api-56bcd9ffd7	1 desired, 1 ready
ReplicaSet	streamlit-c8df87dbd	1 desired, 1 ready
ReplicaSet	streamlit-6c678779c7	0 desired (previous revision)
Pod	flask-api-56bcd9ffd7-qgf9r	Running, 0 restarts

Resource Type	Name	Status
Pod	streamlit-c8df87dbd-pw8s8	Running, 0 restarts
Service	flask-api-service	ClusterIP, 5000/TCP
Service	streamlit-service	NodePort, 8501:31883/TCP

16.4 Prediction Results

Table 42: Prediction results

Interface	Image	Predicted Class	Confidence
Flask API via curl	frog.png	Frog	99.01%
Flask API via curl	truck.png	Truck	86.98%
Streamlit UI	Frog.png	Frog	99.01%

16.5 Evaluation Criteria Assessment

Table 43: Evaluation criteria assessment

Criterion	Assessment
Deployment success	Achieved. Every resource reached its desired state without errors or restarts.
YAML configuration accuracy	Achieved. Selectors, labels, ports, image names and Service names are consistent and the manifests were accepted and worked on the first apply.
Operational understanding	Demonstrated through explanation of Pods, Deployments, Services, ClusterIP and NodePort, imagePullPolicy, service discovery, port-forwarding and ReplicaSet behaviour during a rollout restart.

17. Observations

17.1 Key Observations

1. Minikube provides a convenient local Kubernetes environment for development and testing.
2. Kubernetes Deployments manage application Pods and maintain the desired replica count.
3. Kubernetes Services provide stable network endpoints for application communication.
4. The Flask API was exposed internally using a ClusterIP Service.
5. The Streamlit frontend was exposed using a NodePort Service.
6. Kubernetes service names can be used for communication between Pods.
7. The local Docker images were loaded into the Minikube environment.
8. The `imagePullPolicy: Never` setting allowed Kubernetes to use the locally built images.
9. Pod and Deployment status can be monitored using `kubectl`.
10. Port forwarding can be used to test an internal Kubernetes Service from the local machine.
11. The complete Streamlit-to-Flask-to-CNN workflow was successfully tested.
12. Kubernetes provides a structured approach for managing multiple containerized application components.

17.2 Detailed Discussion

Declarative configuration

The deployment was driven entirely by four short YAML files. Re-applying a file is safe, and the cluster converges on the declared state. This is different from running containers by hand, where the operator is responsible for every start, stop and restart.

Separation of Deployment and Service

Deployments manage what runs; Services manage how it is reached. Because the two are linked only by labels, the Pods behind a Service can be replaced or restarted without changing the address used by clients. This was demonstrated when the Streamlit Pod was recreated by the rollout restart: the Service address 10.106.207.53 and node port 31883 stayed the same.

Service discovery

The Streamlit application contains no IP addresses. It uses the name flask-api-service, which is resolved by cluster DNS. This is the Kubernetes equivalent of the container name networking used in Docker Compose, with the added benefit of a stable virtual IP and load balancing across replicas.

Local image workflow

Building images with `minikube image build` removed the need for a registry. The trade-off is that `imagePullPolicy: Never` must be set, and the images exist only in that cluster; a deployment to another cluster would require pushing the images to a registry and changing the policy.

Resource size

Each application image is roughly 800 MB because TensorFlow is installed in both images from one requirements file. The Streamlit image does not need TensorFlow, so separate requirement files would reduce its size and start-up time.

Stability

Neither Pod restarted at any time (RESTARTS 0 in all snapshots) and the Flask Pod ran continuously for at least 19 minutes. The model loaded successfully on start-up and predictions were returned consistently.

17.3 Limitations of the Current Setup

Table 44: Limitations and improvements

Limitation	Explanation	Possible Improvement
Single replica	One Pod per Deployment means no redundancy	Increase spec.replicas to 2 or more
No health probes	Kubernetes cannot detect a running but unhealthy application	Add readinessProbe and livenessProbe on / for Flask
No resource limits	Pods may use unbounded CPU and memory	Define resources.requests and resources.limits
Model inside the image	A new model needs a new image	Mount the model from a volume or object storage
Local images and Never policy	Works only on the cluster that built the images	Push to a registry and use IfNotPresent
Same requirements for both images	Large images	Separate requirements files
Single node	No node failure tolerance	Use a multi-node cluster in production

17.4 Challenges and How They Were Handled

Table 45: Challenges encountered

Challenge	Solution
Kubernetes cannot pull locally built images from a registry	Built images inside Minikube and set imagePullPolicy: Never
ClusterIP Service is not reachable from the host browser	Used kubectl port-forward for testing
NodePort is not directly reachable with the Docker driver on Windows	Used minikube service streamlit-service --url
Streamlit must find Flask without a fixed IP	Used the Service name flask-api-service as the host name
Updating the running Streamlit application	Used kubectl rollout restart and rollout status

18. Conclusion

This task successfully demonstrated the setup and deployment of a containerized deep learning application using Kubernetes and Minikube.

A local Kubernetes cluster was created and verified. The Flask REST API and the Streamlit frontend were deployed using Kubernetes Deployments. Kubernetes Services were configured to provide internal backend communication and external frontend access.

The Flask API was tested using health checks and CIFAR-10 image predictions: the frog image was classified as Frog with 99.01 percent confidence and the truck image as Truck with 86.98 percent confidence. The Streamlit frontend was also connected to the Flask backend using the Kubernetes Service name, and an end-to-end image prediction workflow was successfully verified.

The implementation provided practical understanding of Kubernetes Pods, Deployments, Services, container images, networking, service discovery and application monitoring. The final system consists of the following structure:

```
Kubernetes / Minikube
        |
        |-- Flask Deployment
        |       `-- Flask API Pod
        |             `-- CIFAR-10 CNN Model
        |
        `-- Streamlit Deployment
                `-- Streamlit Pod
                       |
                       `-- flask-api-service
```

The application demonstrates container orchestration, service-based communication, API integration and deep learning inference using Kubernetes. The same manifests and concepts can be extended to multiple replicas, health probes, resource limits and a multi-node cluster for a production-style deployment.

18.1 Future Scope

- Scale the Flask Deployment to several replicas and observe load balancing through the Service.
- Add readiness and liveness probes so Kubernetes can detect unhealthy Pods.
- Define CPU and memory requests and limits.
- Use ConfigMaps for the API URL instead of hard-coding it in app.py.
- Add an Ingress resource for friendly URL-based access.
- Push the images to a container registry and deploy to a managed cluster.
- Add a Horizontal Pod Autoscaler to scale the API with load.

19. Screenshots

This section collects all fourteen deployment screenshots in the order in which the work was performed. Each screenshot is followed by a short description of what it proves. The YAML editor screenshots are shown in Sections 9 and 10.

19.1 Minikube cluster verification

```
PS D:\Task12_Kubernetes> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

PS D:\Task12_Kubernetes> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   50s   v1.37.0
PS D:\Task12_Kubernetes>
```

Figure 22: Screenshot 1: Minikube cluster verification

minikube status shows host, kubelet and apiserver Running and kubeconfig Configured. kubectl get nodes shows node minikube Ready with role control-plane on version v1.37.0.

19.2 Minikube Docker images

```
PS D:\Task12_Kubernetes> minikube image ls --format table
```

IMAGE	TAG	IMAGE ID	SIZE
registry.k8s.io/etcd	3.7.0-0	sha256:270fbe	22.3MB
docker.io/kindest/kindnetd	v20260820-69b56db7	sha256:4626fe	39.7MB
docker.io/library/task12-streamlit	latest	sha256:b964f6	809MB
gcr.io/k8s-minikube/storage-provisioner	v5	sha256:6e38f4	9.06MB
registry.k8s.io/kube-apiserver	v1.37.0	sha256:bec5f0	31.2MB
registry.k8s.io/kube-controller-manager	v1.37.0	sha256:364b3c	27.3MB
registry.k8s.io/kube-proxy	v1.37.0	sha256:d6a28d	30.2MB
registry.k8s.io/kube-scheduler	v1.37.0	sha256:1fabf8	20.8MB
registry.k8s.io/pause	3.10.2	sha256:4a83b1	321kB
docker.io/kindest/kindnetd	v20250512-df8de77b	sha256:409467	44.4MB
docker.io/library/task12-flask-api	latest	sha256:92255d	813MB
registry.k8s.io/coredns/coredns	v1.14.6	sha256:520212	23.8MB

```
PS D:\Task12_Kubernetes>
```

Figure 23: Screenshot 2: Minikube Docker images

The images task12-flask-api:latest (813MB) and task12-streamlit:latest (809MB) are available inside Minikube together with the Kubernetes system images.

19.3 Flask Pod running

```
PS D:\Task12_Kubernetes> kubectl get pods
NAME                         READY   STATUS    RESTARTS   AGE
flask-api-56bcd9ffd7-qgf9r   1/1     Running   0          44s
PS D:\Task12_Kubernetes>
```

Figure 24: Screenshot 3: Flask Pod running

Pod flask-api-56bcd9ffd7-qgf9r is Running, 1/1 ready, 0 restarts, age 44s.

19.4 Kubernetes Pods running

```
PS D:\Task12_Kubernetes> kubectl get pods
NAME                         READY   STATUS    RESTARTS   AGE
flask-api-56bcd9ffd7-qgf9r   1/1     Running   0          2m24s
streamlit-6c678779c7-v4hjd   1/1     Running   0          43s
PS D:\Task12_Kubernetes>
```

Figure 25: Screenshot 4: Kubernetes Pods running

Both flask-api and streamlit Pods are Running with 1/1 ready and 0 restarts.

19.5 Kubernetes Services

```
PS D:\Task12_Kubernetes> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
flask-api-service   ClusterIP   10.109.95.216   <none>        5000/TCP         3m11s
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          53m
streamlit-service   NodePort    10.106.207.53   <none>        8501:31883/TCP   90s
PS D:\Task12_Kubernetes>
```

Figure 26: Screenshot 5: Kubernetes Services

flask-api-service (ClusterIP, 5000/TCP), kubernetes (ClusterIP, 443/TCP) and streamlit-service (NodePort, 8501:31883/TCP).

19.6 Kubernetes Streamlit application

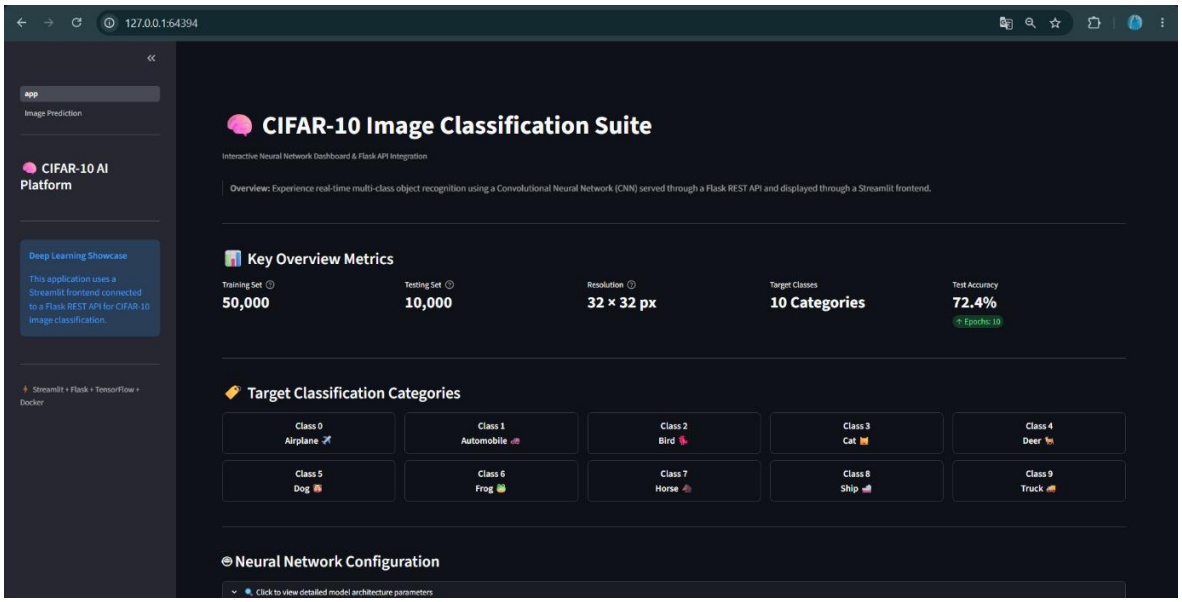


Figure 27: Screenshot 6: Kubernetes Streamlit application

The Streamlit dashboard served from the Pod in Kubernetes through the Minikube service URL.

19.7 Kubernetes Flask API health

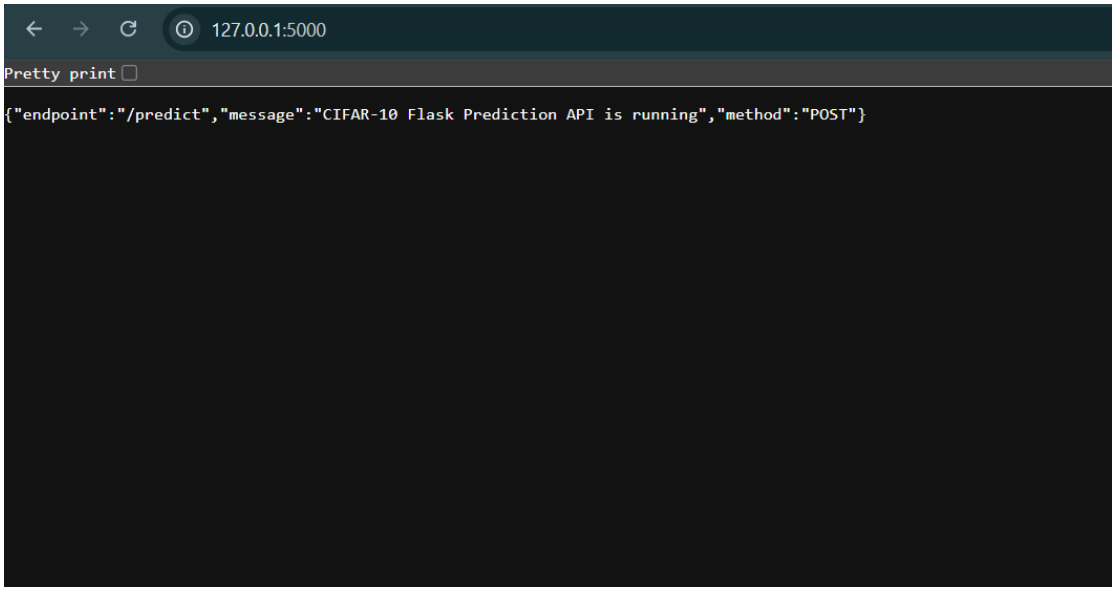


Figure 28: Screenshot 7: Kubernetes Flask API health

The Flask health endpoint reached through kubectl port-forward returns the API running message.

19.8 Kubernetes frog prediction

```
PS D:\Task12_Kubernetes> cd D:\Task12_Kubernetes
PS D:\Task12_Kubernetes> curl.exe -X POST -F "image=@frog.png" http://127.0.0.1:5000/predict
{"confidence":99.01,"predicted_class":"Frog"}
PS D:\Task12_Kubernetes>
```

Figure 29: Screenshot 8: Kubernetes frog prediction

curl POST of frog.png returns Frog with confidence 99.01.

19.9 Kubernetes truck prediction

```
PS D:\Task12_Kubernetes> cd D:\Task12_Kubernetes
PS D:\Task12_Kubernetes> curl.exe -X POST -F "image=@truck.png" http://127.0.0.1:5000/predict
{"confidence":86.98,"predicted_class":"Truck"}
PS D:\Task12_Kubernetes>
```

Figure 30: Screenshot 9: Kubernetes truck prediction

curl POST of truck.png returns Truck with confidence 86.98.

19.10 Kubernetes deployments

```
PS D:\Task12_Kubernetes> kubectl get deployments
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
flask-api   1/1     1            1           10m
streamlit   1/1     1            1           9m13s
PS D:\Task12_Kubernetes>
```

Figure 31: Screenshot 10: Kubernetes deployments

Deployments flask-api and streamlit show 1/1 ready, 1 up-to-date and 1 available.

19.11 Kubernetes cluster resources

```
PS D:\Task12_Kubernetes> kubectl get pods,services,deployments
NAME                                 READY   STATUS    RESTARTS   AGE
pod/flask-api-56bcd9ffd7-qgf9r       1/1     Running   0          11m
pod/streamlit-6c678779c7-v4hjd       1/1     Running   0          9m56s

NAME                           TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/flask-api-service      ClusterIP   10.109.95.216   <none>        5000/TCP         11m
service/kubernetes             ClusterIP   10.96.0.1       <none>        443/TCP          62m
service/streamlit-service      NodePort    10.106.207.53   <none>        8501:31883/TCP   9m49s

NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-api            1/1     1            1           11m
deployment.apps/streamlit            1/1     1            1           9m56s
PS D:\Task12_Kubernetes>
```

Figure 32: Screenshot 11: Kubernetes cluster resources

kubectl get pods,services,deployments shows all resources healthy.

19.12 Streamlit Flask API connected



Figure 33: Screenshot 12: Streamlit Flask API connected

The Check Flask API Connection button reports that the Flask API is connected and running.

19.13 Kubernetes Streamlit frog prediction

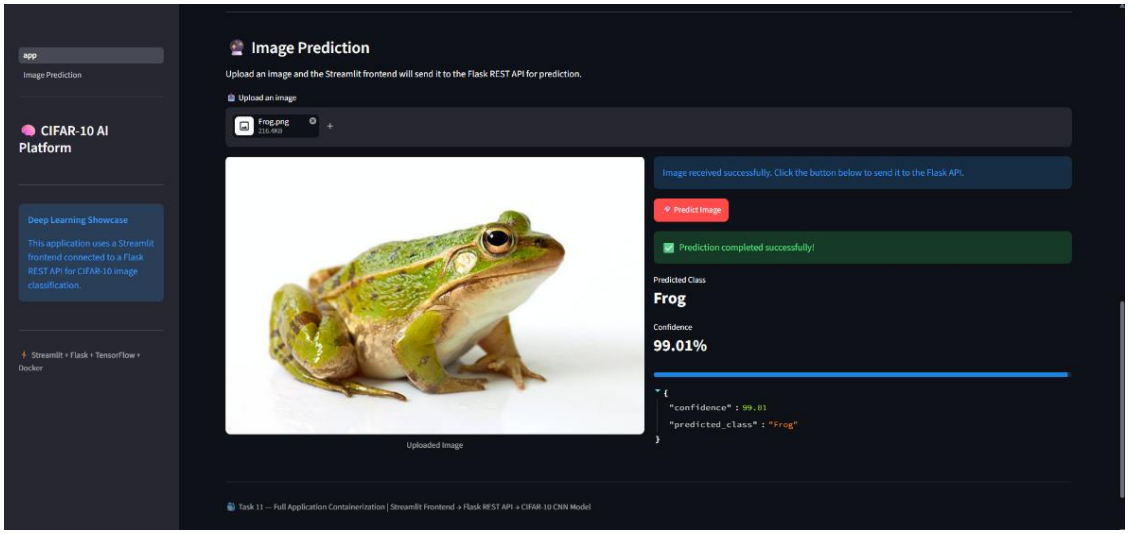


Figure 34: Screenshot 13: Kubernetes Streamlit frog prediction

Prediction made in the Streamlit UI: Frog with 99.01% confidence.

19.14 Final Kubernetes verification

```
PS D:\Task12_Kubernetes> kubectl get all
NAME                                 READY   STATUS    RESTARTS   AGE
pod/flask-api-56bcd9ffd7-qgf9r       1/1     Running   0          19m
pod/streamlit-c8df87dbd-pw8s8        1/1     Running   0          3m29s

NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/flask-api-service   ClusterIP   10.109.95.216   <none>        5000/TCP         19m
service/kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          70m
service/streamlit-service   NodePort    10.106.207.53   <none>        8501:31883/TCP   18m

NAME                        READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-api   1/1     1            1           19m
deployment.apps/streamlit   1/1     1            1           18m

NAME                                   DESIRED   CURRENT   READY   AGE
replicaset.apps/flask-api-56bcd9ffd7   1         1         1       19m
replicaset.apps/streamlit-6c678779c7   0         0         0       18m
replicaset.apps/streamlit-c8df87dbd    1         1         1       3m29s
PS D:\Task12_Kubernetes>
```

Figure 35: Screenshot 14: Final Kubernetes verification

kubectl get all lists Pods, Services, Deployments and ReplicaSets, including the old and new Streamlit ReplicaSets.

20. Appendix

The appendix reproduces the complete source files of the project exactly as submitted.

Table 46: Appendix contents

Appendix	File
A	flask_api.py
B	app.py
C	pages/1_Image_Prediction.py
D	Kubernetes manifests (four files)
E	Docker files, docker-compose.yml, requirements.txt and .dockerignore
F	Command reference
G	Glossary

Appendix A: flask_api.py

Flask REST API that loads the CNN model and serves the / and /predict endpoints.

flask_api.py

```
from flask import Flask, request, jsonify
import tensorflow as tf
import numpy as np
from PIL import Image
import io
# ========================================
# Flask Application
# ========================================
app = Flask(__name__)
# ========================================
# CIFAR-10 Class Names
# ========================================
class_names = [
    "Airplane",
    "Automobile",
    "Bird",
    "Cat",
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck"
]
# ========================================
# Load Trained CNN Model
# ========================================
MODEL_PATH = "model/cifar10_cnn.keras"
model = tf.keras.models.load_model(MODEL_PATH)
# ========================================
# Home / Health Check Endpoint
# ========================================
@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "CIFAR-10 Flask Prediction API is running",
        "endpoint": "/predict",
        "method": "POST"
    })
# ========================================
# Prediction Endpoint
# ========================================
```

```
@app.route("/predict", methods=["POST"])
def predict():
    # Check whether image was provided
    if "image" not in request.files:
        return jsonify({
            "error": "No image provided. Please upload an image."
        }), 400
    try:
        # Get uploaded image
        image_file = request.files["image"]
        # Open image and convert to RGB
        image = Image.open(image_file).convert("RGB")
        # Resize image to CIFAR-10 input size
        image = image.resize((32, 32))
        # Convert image to NumPy array
        image_array = np.array(image)
        # Normalize pixel values
        image_array = image_array.astype("float32") / 255.0
        # Add batch dimension
        image_array = np.expand_dims(image_array, axis=0)
        # Make prediction
        predictions = model.predict(
            image_array,
            verbose=0
        )
        # Find predicted class
        predicted_index = np.argmax(predictions[0])
        predicted_class = class_names[predicted_index]
        # Calculate confidence
        confidence = float(predictions[0][predicted_index]) * 100
        # Return JSON response
        return jsonify({
            "predicted_class": predicted_class,
            "confidence": round(confidence, 2)
        })
    except Exception as e:
        return jsonify({
            "error": str(e)
        }), 500
# ========================================
# Run Flask Application
# ========================================
if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False
    )
```

Appendix B: app.py

Streamlit dashboard with the backend status check and image prediction.

app.py

```
import streamlit as st
import requests
from PIL import Image
# ========================================
# Page Configuration & Styling
# ========================================
st.set_page_config(
    page_title="CIFAR-10 Deep Learning App",
    page_icon="🧠",
    layout="wide"
)
# ========================================
# UI Styling
# ========================================
st.markdown(
    """
    <style>
    [data-testid="stMetricValue"] {
        font-size: 28px;
        font-weight: 700;
    }
    .class-badge {
        border: 1px solid #313745;
        border-radius: 8px;
        padding: 10px;
        text-align: center;
        font-weight: 600;
        margin-bottom: 10px;
    }
    .class-badge:hover {
        border-color: #4CAF50;
    }
    [data-testid="stHeader"] {
        display: none !important;
    }
    #MainMenu {
        visibility: hidden !important;
    }
    footer {
        visibility: hidden !important;
    }
    header {
        visibility: hidden !important;
    }
    .stApp > header {
        display: none !important;
    }
    button[title="View fullscreen"] {
        display: none !important;
    }
    [data-testid="stElementToolbar"] {
        display: none !important;
    }
    [data-testid="stStyledFullScreenButton"] {
        display: none !important;
    }
    [data-testid="stStatusWidget"] {
        display: none !important;
    }
    </style>
    """,
    unsafe_allow_html=True
)
# ========================================
# Flask API Configuration
```

```
# ========================================
# IMPORTANT:
# "flask-api" will be the Flask service name
# inside docker-compose.yml.
FLASK_API_URL = "http://flask-api-service:5000"
# ========================================
# CIFAR-10 Class Names
# ========================================
class_names = [
    "Airplane ✈️",
    "Automobile 🚗",
    "Bird 🐦",
    "Cat 🐱",
    "Deer 🦌",
    "Dog 🐶",
    "Frog 🐸",
    "Horse 🐴",
    "Ship 🚢",
    "Truck 🚚"
]
# ========================================
# Sidebar
# ========================================
st.sidebar.title("🧠 CIFAR-10 AI Platform")
st.sidebar.markdown("---")
st.sidebar.info(
    "**Deep Learning Showcase**\n\n"
    "This application uses a Streamlit frontend "
    "connected to a Flask REST API for CIFAR-10 image classification."
)
st.sidebar.markdown("---")
st.sidebar.caption("⚡ Streamlit + Flask + TensorFlow + Docker")
# ========================================
# Hero Banner
# ========================================
st.title("🧠 CIFAR-10 Image Classification Suite")
st.caption(
    "Interactive Neural Network Dashboard & Flask API Integration"
)
st.markdown(
    """
    > **Overview:** Experience real-time multi-class object recognition
    using a Convolutional Neural Network (CNN) served through a Flask
    REST API and displayed through a Streamlit frontend.
    """
)
st.markdown("---")
# ========================================
# Key Dataset Metrics
# ========================================
st.subheader("📊 Key Overview Metrics")
m1, m2, m3, m4, m5 = st.columns(5)
with m1:
    st.metric(
        "Training Set",
        "50,000",
        help="Number of images used during model training"
    )
with m2:
    st.metric(
        "Testing Set",
        "10,000",
        help="Independent images used for evaluation"
    )
with m3:
    st.metric(
        "Resolution",
        "32 × 32 px",
        help="RGB Color Input Dimensions"
    )
with m4:
    st.metric(
        "Target Classes",
        "10 Categories"
```

```
    )
with m5:
    st.metric(
        "Test Accuracy",
        "72.4%",
        delta="Epochs: 10"
    )
st.markdown("---")
# ========================================
# Class Categories
# ========================================
st.subheader("🏷️ Target Classification Categories")
cols = st.columns(5)
for idx, class_name in enumerate(class_names):
    with cols[idx % 5]:
        st.markdown(
            f"""
            <div class="class-badge">
                Class {idx}<br>
                <b>{class_name}</b>
            </div>
            """,
            unsafe_allow_html=True
        )
st.markdown("---")
# ========================================
# Neural Network Configuration
# ========================================
st.subheader("🤖 Neural Network Configuration")
with st.expander(
    "🔍 Click to view detailed model architecture parameters",
    expanded=True
):
    col_a, col_b = st.columns(2)
    with col_a:
        st.markdown(
            """
            * **Architecture:** Sequential Convolutional Neural Network (CNN)
            * **Optimization Algorithm:** Adam
            * **Loss Calculation:** Sparse Categorical Crossentropy
            * **Classification Layer:** 10 Units with Softmax Activation
            """
        )
    with col_b:
        st.markdown(
            """
            * **Input Tensor Shape:** `(32, 32, 3)`
            * **Feature Extractors:** 2x Conv2D + MaxPooling Blocks
            * **Regularization:** Dropout (0.5 Rate)
            * **Hidden Layer Units:** 128 Dense Units (ReLU Activation)
            """
        )
st.markdown("---")
# ========================================
# Flask API Connection Check
# ========================================
st.subheader("🔌 Backend API Status")
if st.button("🔄 Check Flask API Connection"):
    try:
        response = requests.get(
            f"{FLASK_API_URL}/",
            timeout=5
        )
        if response.status_code == 200:
            st.success(
                "✅ Flask API is connected and running."
            )
            st.json(response.json())
        else:
            st.error(
                f"⚠️ Flask API returned status code "
                f"{response.status_code}"
            )
    except requests.exceptions.RequestException as e:
        st.error(
```

```
            "❌ Unable to connect to Flask API."
        )
        st.caption(
            f"Connection details: {e}"
        )
# ========================================
# Image Prediction
# ========================================
st.markdown("---")
st.subheader("🔮 Image Prediction")
st.write(
    "Upload an image and the Streamlit frontend will send it "
    "to the Flask REST API for prediction."
)
uploaded_file = st.file_uploader(
    "📤 Upload an image",
    type=["jpg", "jpeg", "png"]
)
if uploaded_file is not None:
    # Display uploaded image
    image = Image.open(uploaded_file).convert("RGB")
    col1, col2 = st.columns(2)
    with col1:
        st.image(
            image,
            caption="Uploaded Image",
            use_container_width=True
        )
    with col2:
        st.info(
            "Image received successfully. "
            "Click the button below to send it to the Flask API."
        )
        predict_button = st.button(
            "🚀 Predict Image",
            type="primary"
        )
        if predict_button:
            try:
                # Reset file position
                uploaded_file.seek(0)
                # Prepare image for Flask API
                files = {
                    "image": (
                        uploaded_file.name,
                        uploaded_file,
                        uploaded_file.type
                    )
                }
                # Send image to Flask API
                response = requests.post(
                    f"{FLASK_API_URL}/predict",
                    files=files,
                    timeout=60
                )
                # ========================================
                # Successful Response
                # ========================================
                if response.status_code == 200:
                    result = response.json()
                    predicted_class = result.get(
                        "predicted_class",
                        "Unknown"
                    )
                    confidence = result.get(
                        "confidence",
                        0
                    )
                    st.success(
                        "✅ Prediction completed successfully!"
                    )
                    st.metric(
                        "Predicted Class",
                        predicted_class
                    )
```

```
                    st.metric(
                        "Confidence",
                        f"{confidence:.2f}%"
                    )
                    st.progress(
                        min(max(confidence / 100, 0.0), 1.0)
                    )
                    st.json(result)
                # ========================================
                # Flask Error Response
                # ========================================
                else:
                    try:
                        error_data = response.json()
                    except Exception:
                        error_data = {
                            "error": response.text
                        }
                    st.error(
                        f"❌ Flask API error "
                        f"(HTTP {response.status_code})"
                    )
                    st.json(error_data)
            except requests.exceptions.RequestException as e:
                st.error(
                    "❌ Could not connect to Flask API."
                )
                st.warning(
                    "Make sure the Flask container is running "
                    "and both containers are connected through "
                    "the Docker network."
                )
                st.caption(
                    f"Connection details: {e}"
                )
            except Exception as e:
                st.error(
                    "❌ An unexpected error occurred."
                )
                st.caption(
                    f"Error details: {e}"
                )
# ========================================
# Footer / Status
# ========================================
st.markdown("---")
st.caption(
    "🐬 Task 11 — Full Application Containerization | "
    "Streamlit Frontend → Flask REST API → CIFAR-10 CNN Model"
)
```

Appendix C: pages/1_Image_Prediction.py

Second Streamlit page that provides a dedicated image prediction view. It uses http://127.0.0.1:5000 as the API address, which suits local execution next to flask_api.py.

1_Image_Prediction.py

```
import streamlit as st
import requests
from PIL import Image
# ========================================
# Page Configuration
# ========================================
st.set_page_config(
    page_title="Image Prediction",
    page_icon="🔮",
    layout="wide"
)
# ========================================
# Flask API Configuration
# ========================================
FLASK_API_URL = "http://127.0.0.1:5000/predict"
# ========================================
# Page Title
# ========================================
st.title("🔮 CIFAR-10 Image Prediction")
st.write(
    "Upload an image and the Streamlit frontend "
    "will send it to the Flask backend for real-time prediction."
)
# ========================================
# Backend Status
# ========================================
try:
    response = requests.get(
        "http://127.0.0.1:5000/",
        timeout=3
    )
    if response.status_code == 200:
        st.success("🟢 Flask API is connected and running.")
    else:
        st.warning("🟡 Flask API responded with an unexpected status.")
except requests.exceptions.RequestException:
    st.error(
        "🔴 Flask API is not running. "
        "Please start flask_api.py first."
    )
# ========================================
# File Uploader
# ========================================
uploaded_file = st.file_uploader(
    "📤 Upload an image",
    type=["jpg", "jpeg", "png"]
)
# ========================================
# Prediction
# ========================================
if uploaded_file is not None:
    # Open image for displaying in Streamlit
    image = Image.open(uploaded_file).convert("RGB")
    # Display uploaded image
    col1, col2 = st.columns(2)
    with col1:
        st.subheader("🖼️ Uploaded Image")
        st.image(
            image,
            caption="Input Image",
            width=300
        )
    # ========================================
```

```
    # Send Image to Flask API
    # ========================================
    with st.spinner("🤖 Sending image to Flask API..."):
        try:
            # Move file pointer to beginning
            uploaded_file.seek(0)
            # Prepare image for HTTP request
            files = {
                "image": (
                    uploaded_file.name,
                    uploaded_file.getvalue(),
                    uploaded_file.type
                )
            }
            # Send POST request to Flask backend
            response = requests.post(
                FLASK_API_URL,
                files=files,
                timeout=30
            )
            # ========================================
            # Process Flask Response
            # ========================================
            if response.status_code == 200:
                result = response.json()
                predicted_class = result.get(
                    "predicted_class",
                    "Unknown"
                )
                confidence = result.get(
                    "confidence",
                    0
                )
                # ========================================
                # Display Prediction
                # ========================================
                with col2:
                    st.subheader("🤖 Prediction")
                    st.success(
                        f"Predicted Class: **{predicted_class}**"
                    )
                    st.metric(
                        "Confidence",
                        f"{confidence:.2f}%"
                    )
                # ========================================
                # API Response
                # ========================================
                st.markdown("---")
                st.subheader("🔬 Flask API Response")
                st.json(result)
                # ========================================
                # Integration Status
                # ========================================
                st.success(
                    "✅ Streamlit → Flask → CNN Model → "
                    "Prediction workflow completed successfully."
                )
            else:
                st.error(
                    f"Flask API returned HTTP "
                    f"{response.status_code}"
                )
                try:
                    st.json(response.json())
                except Exception:
                    st.text(response.text)
        except requests.exceptions.ConnectionError:
            st.error(
                "❌ Could not connect to Flask API. "
                "Make sure flask_api.py is running on port 5000."
            )
        except requests.exceptions.Timeout:
            st.error(
                "⏱️ Flask API request timed out. "
```

```
                "Please try again."
            )
        except Exception as e:
            st.error(
                f"❌ An unexpected error occurred: {str(e)}"
            )
else:
    st.info(
        "👆 Please upload a JPG, JPEG, or PNG image to begin."
    )
```

Appendix D: Kubernetes Manifests

flask-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-api
  template:
    metadata:
      labels:
        app: flask-api
    spec:
      containers:
        - name: flask-api
          image: task12-flask-api:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
```

flask-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
spec:
  selector:
    app: flask-api
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: ClusterIP
```

streamlit-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: streamlit
spec:
  replicas: 1
  selector:
    matchLabels:
      app: streamlit
  template:
    metadata:
      labels:
        app: streamlit
    spec:
      containers:
        - name: streamlit
          image: task12-streamlit:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8501
```

streamlit-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: streamlit-service
spec:
```

```
  selector:
    app: streamlit
  ports:
    - protocol: TCP
      port: 8501
      targetPort: 8501
  type: NodePort
```

Appendix E: Container Files

Dockerfile.flask

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY flask_api.py .
COPY model ./model
EXPOSE 5000
CMD ["python", "flask_api.py"]
```

Dockerfile.streamlit

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY pages ./pages
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

docker-compose.yml

```
services:
  flask-api:
    build:
      context: .
      dockerfile: Dockerfile.flask
    container_name: task11-flask-api
    ports:
      - "5000:5000"
    networks:
      - task11-network
  streamlit:
    build:
      context: .
      dockerfile: Dockerfile.streamlit
    container_name: task11-streamlit
    ports:
      - "8501:8501"
    depends_on:
      - flask-api
    networks:
      - task11-network
networks:
  task11-network:
    driver: bridge
```

requirements.txt

```
flask
streamlit
requests
tensorflow
numpy
pillow
```

.dockerignore

```
__pycache__
```

```
*.pyc
*.pyo
*.pyd
.git
.gitignore
.vscode
screenshots
screenshots9
README.md
```

The docker-compose.yml file belongs to the previous containerization task and runs both containers on a Docker bridge network. It is kept in the project folder for comparison and is not used by Kubernetes.

Appendix F: Command Reference

Table 47: Commands used in the project

Command	Purpose
minikube start --driver=docker	Create and start the local cluster
minikube status	Check cluster component status
kubectl get nodes	List nodes and verify Ready state
minikube image build -t task12-flask-api:latest -f Dockerfile.flask .	Build the Flask image inside Minikube
minikube image build -t task12-streamlit:latest -f Dockerfile.streamlit .	Build the Streamlit image inside Minikube
minikube image ls --format table	List images in Minikube
kubectl apply -f flask-deployment.yaml	Create the Flask Deployment
kubectl apply -f flask-service.yaml	Create the Flask Service
kubectl apply -f streamlit-deployment.yaml	Create the Streamlit Deployment
kubectl apply -f streamlit-service.yaml	Create the Streamlit Service
kubectl get pods	List Pods
kubectl get services	List Services
kubectl get deployments	List Deployments
kubectl get pods,services,deployments	List the three resource types together
kubectl get all	List all common resources including ReplicaSets
kubectl port-forward service/flask-api-service 5000:5000	Expose the Flask Service on local port 5000
minikube service streamlit-service --url	Obtain a local URL for the Streamlit NodePort Service
kubectl rollout restart deployment streamlit	Restart Streamlit Pods through a new ReplicaSet
kubectl rollout status deployment streamlit	Follow the rollout progress
curl.exe -X POST -F "image=@frog.png" http://127.0.0.1:5000/predict	Send the frog image to the API
curl.exe -X POST -F "image=@truck.png" http://127.0.0.1:5000/predict	Send the truck image to the API

Appendix G: Glossary

Table 48: Glossary of terms

Term	Definition
Cluster	A set of machines (here one node) managed by Kubernetes
Node	A machine in the cluster that runs Pods
Control plane	Components that manage the cluster: API server, scheduler, controller manager, etcd
Pod	Smallest deployable unit; one or more containers sharing network and storage
ReplicaSet	Keeps a specified number of identical Pods running
Deployment	Manages ReplicaSets and provides declarative updates for Pods
Service	Stable network endpoint and DNS name for a set of Pods
ClusterIP	Service type reachable only inside the cluster
NodePort	Service type that opens a port on every node
kubectl	Command-line client for the Kubernetes API
imagePullPolicy	Rule that decides whether the node pulls an image from a registry
Minikube	Tool that creates a local single-node Kubernetes cluster
CNN	Convolutional Neural Network used for image classification
kubelet	Node agent that starts and monitors Pods
kube-proxy	Programs forwarding rules that implement Services
CoreDNS	Cluster DNS server for Service names
etcd	Key-value store with all cluster state
Rollout	Process of replacing Pods when a Deployment changes
Manifest	YAML file describing a Kubernetes object
Label / Selector	Key-value tags on objects and the queries that match them
REST API	HTTP interface for requesting predictions
CIFAR-10	Dataset of 60,000 32 x 32 color images in 10 classes